



 @azdocuments

INTRODUCTION TO DBMS

SUBJECT CODE: BCS755A

MODULE-1

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

INTRODUCTION TO DBMS

BCS755A

MODULE-1

Introduction to Databases: Introduction, Characteristics of database approach, Advantages of using the DBMS approach, History of database applications.

Overview of Database Languages and Architectures: Data Models, Schemas, and Instances. Three schema architecture and data independence, database languages, and interfaces, The Database System environment.

Textbook 1:Ch 1.1 to 1.8, 2.1 to 2.6

1.1 Introduction

A database is a collection of related data.

A data mean known facts that can be recorded and that have implicit meaning. For eg, consider the names, telephone numbers, and addresses of the people you know. You may have recorded this data in an indexed address book or you may have stored it on a hard drive, using a personal computer and software such as Microsoft Access or Excel. This collection of related data with an implicit meaning is a database.

A database has the following implicit properties:

- A database represents some aspect of the real world, sometimes called the mini world or the universe of discourse (UoD). Changes to the mini world are reflected in the database.
- A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.
- A database is designed, built, and populated with data for a specific purpose. It has an intended group of users and some preconceived applications in which these users are interested.

Other properties:

- A database can be of any size and complexity. For example, the list of names and addresses referred to earlier may consist of only a few hundred records, each with a simple structure. An example of a large commercial database is Amazon.com.
- A database may be generated and maintained manually or it may be computerized. For example, a library card catalog is a database that may be created and maintained manually. A computerized database may be created and maintained either by a group of application programs written specifically for that task or by a database management system.

A database management system (DBMS) is a collection of programs that enables users to create and maintain a database.

The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.

Defining a database involves specifying the data types, structures, and constraints of the data to be stored in the database. The database definition or descriptive information is also stored by the DBMS in the form of a database catalog or dictionary; it is called meta-data.

- Constructing the database is the process of storing the data on some storage medium that is controlled by the DBMS. Manipulating a database includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the miniworld, and generating reports from the data.
- Sharing a database allows multiple users and programs to access the database simultaneously.
- An application program accesses the database by sending queries or requests for data to the DBMS.
- A query typically causes some data to be retrieved; a transaction may cause some data to be read and some data to be written into the database.
- Other important functions provided by the DBMS include protecting the database and maintaining it over a long period of time.
- Protection includes system protection against hardware or software malfunction (or crashes) and security protection against unauthorized or malicious access.

- A typical large database may have a life cycle of many years, so the DBMS must be able to maintain the database system by allowing the system to evolve as requirements change over time.

Database and DBMS software together are a Database system.

A simplified database system environment.

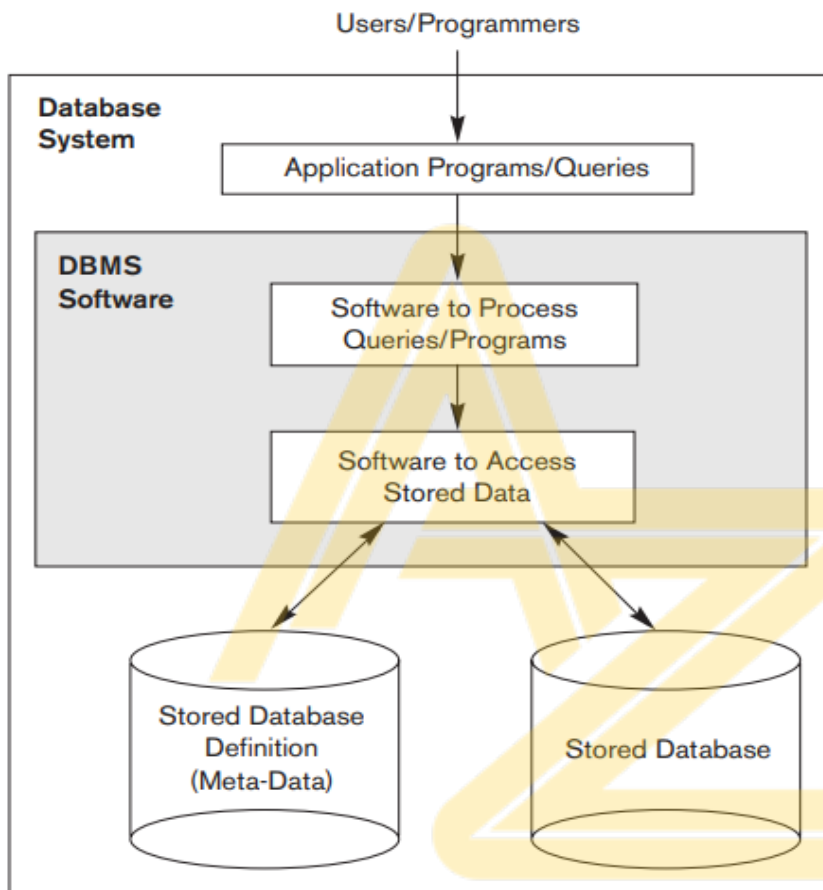


Figure 1.1
A simplified database system environment.

1.2 An Example (University Database)

Introduction

To understand how a Database Management System (DBMS) works, consider a simple **UNIVERSITY database**. This database is designed to store and manage information related to students, courses, instructors, grades, and prerequisites. Instead of maintaining separate files for every activity, all related information is stored in an organized database so that it can be easily accessed, updated, and managed. This example demonstrates how real-world data is represented inside a database and how different pieces of information are connected with one another.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2

A database that stores student and course information.

Organization of the UNIVERSITY Database

The UNIVERSITY database is organized into **five different files (relations/tables)**. Each file stores records of a particular type of entity. A record represents one instance of an entity, while the columns represent different attributes of that entity.

The five files are:

- **STUDENT** – Stores information about every student.
- **COURSE** – Stores details of all courses offered by the university.
- **SECTION** – Stores information about different sections of a course offered in a particular semester.
- **GRADE_REPORT** – Stores the grades obtained by students in different course sections.
- **PREREQUISITE** – Stores prerequisite relationships between courses.

Together, these files represent the complete academic information maintained by the university.

Structure of Records

Before storing data, the structure of each record must be clearly defined. This involves specifying the attributes that each record should contain and assigning an appropriate data type to every attribute.

For example, the **STUDENT** relation contains the following attributes:

- Name
- Student_Number
- Class
- Major

Similarly, the **COURSE** relation stores:

- Course_Name
- Course_Number
- Credit_Hours
- Department

Every record in these relations follows the same structure, ensuring consistency throughout the database.

Data Types

Each attribute in the database is associated with a suitable data type. Selecting appropriate data types helps the DBMS store and process information efficiently.

For example,

- **Name** is stored as a character string.
- **Student_Number** is stored as an integer.
- **Grade** is stored as a single character such as A, B, C, D, F or I.
- **Credit_Hours** is stored as an integer.

Using appropriate data types improves storage efficiency and maintains data integrity.

Coding Scheme

In many databases, actual values are represented using codes to save storage space and simplify processing. This technique is known as a **coding scheme**.

For example, instead of storing the complete class names, the UNIVERSITY database uses numerical codes.

Code Meaning

- | | |
|---|------------------|
| 1 | Freshman |
| 2 | Sophomore |
| 3 | Junior |
| 4 | Senior |
| 5 | Graduate Student |

Such coding schemes reduce redundancy and improve processing speed.

Relationship Between Records

The records stored in different tables are not independent; they are related to one another.

For example, a student appearing in the **STUDENT** table is linked to multiple records in the **GRADE_REPORT** table, which store the grades obtained by that student.

Similarly, the **PREREQUISITE** table establishes relationships between different courses by specifying which course must be completed before enrolling in another course.

These relationships allow the database to retrieve meaningful information from multiple tables simultaneously.

Database Manipulation

Once the database has been created, users interact with it through **database manipulation operations**. These operations mainly include **querying** and **updating**.

Query Operations

A **query** is a request to retrieve information from the database. Queries do not modify the stored data; they only extract the required information.

Some examples include:

- Retrieving the transcript of a particular student.
- Listing all students enrolled in a specific course.
- Displaying the prerequisites of a course.

Queries enable users to obtain meaningful information quickly without manually searching through the database.

Update Operations

An **update** modifies the existing contents of the database. Updates ensure that the database always reflects the current state of the real world.

Examples of update operations include:

- Changing a student's class from Freshman to Sophomore.
- Creating a new section for a course.
- Entering or modifying a student's grade.

Updating is an essential operation because university information changes continuously.

Query Language

Although users express their requirements in natural language, the DBMS cannot understand ordinary English directly. Therefore, all queries and updates must be written using a **database query language**, such as **Structured Query Language (SQL)**.

The DBMS interprets these commands, accesses the required records, performs the requested operation, and returns the desired result.

Database as Part of an Information System

A database does not work independently. It is one component of a larger **Information System** within an organization.

The Information Technology (IT) department is responsible for designing, developing, and maintaining this information system. Apart from the database, the system also consists of computers, storage devices, application software, users, and networking infrastructure.

The database acts as the central repository from which different applications retrieve and update information.

Database Design Process

The development of a database follows a systematic design process.

1. Requirements Specification and Analysis

The first step is to collect and analyze the requirements of the organization. During this phase, designers identify what information needs to be stored, who will use the database, and what operations will be performed.

2. Conceptual Design

The collected requirements are converted into a conceptual model using the **Entity-Relationship (ER) Model**. This phase identifies entities, attributes, and relationships without considering implementation details.

3. Logical Design

The conceptual model is then converted into a logical database schema based on a suitable data model, such as the relational model. Tables, keys, and relationships are defined during this stage.

4. Physical Design

In this phase, decisions are made regarding how the database will actually be stored on disk. Storage structures, indexing techniques, and access methods are selected to improve performance.

5. Database Implementation and Maintenance

Finally, the database is implemented in a commercial DBMS, populated with actual data, and continuously maintained. Maintenance includes inserting new records, updating existing information, deleting obsolete data, performing backups, and ensuring security.

- The UNIVERSITY database consists of five relations: **STUDENT, COURSE, SECTION, GRADE_REPORT, and PREREQUISITE**.
- Every relation contains records with predefined attributes and data types.
- Related records are connected through relationships.
- Database manipulation includes **query** and **update** operations.
- SQL is used to communicate with the DBMS.
- Database development follows the phases: **Requirements Analysis → Conceptual Design → Logical Design → Physical Design → Implementation → Maintenance**.

1.3 Characteristics of the Database Approach

Introduction

The database approach was developed to overcome the limitations of the traditional file processing system. In the file processing approach, every department or application maintains its own separate files and programs. As a result, the same information is often stored multiple times, leading to **data redundancy**, **data inconsistency**, increased storage requirements, and difficulty in maintaining up-to-date information.

The **Database Management System (DBMS)** eliminates these problems by maintaining a **single integrated database** that can be accessed by multiple users and applications. Instead of creating separate files for every application, all data is stored in one centralized repository and managed by the DBMS. This allows different users to access the same data through queries, transactions, and application programs while ensuring consistency and security.

The database approach is distinguished from the traditional file processing system by four major characteristics:

1. Self-describing nature of a database system
2. Insulation between programs and data (Data Abstraction)
3. Support for multiple views of data
4. Sharing of data and multiuser transaction processing

1. Self-Describing Nature of a Database System

One of the most important characteristics of a DBMS is that it is **self-describing**. This means that a database system stores not only the actual data but also a complete description of the database structure. This description is known as **metadata**, which is stored in a special repository called the **DBMS Catalog (Data Dictionary)**.

The catalog contains information such as:

- Structure of each table (relation)
- Data types of attributes
- Storage format of data
- Constraints on data
- Relationships between tables

Whenever the DBMS receives a request to access data, it first consults the catalog to understand the structure of the database before retrieving the required information. Therefore, the DBMS software is **independent of any specific database application** and can work with different databases such as banking, hospital, university, or library databases without changing its internal software.

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

Figure 1.3

An example of a database catalog for the database in Figure 1.2.

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerated type with all known majors.

XXXXNNNN is used to define a type with four alphabetic characters followed by four numeric digits.

In contrast, in the traditional file processing system, the structure of files is embedded directly into application programs. Consequently, each application is designed to work only with one specific file structure, making it inflexible and difficult to maintain.

2. Insulation Between Programs and Data (Data Abstraction)

In the traditional file processing approach, application programs are tightly coupled with the structure of data files. If the structure of a file changes, every program that accesses the file must also be modified. This increases maintenance cost and reduces flexibility.

A DBMS overcomes this limitation through **program-data independence**, where the database structure is stored separately in the catalog instead of being embedded in application programs. If a new attribute, such as **Birth_Date**, is added to the STUDENT table, only the database definition in the catalog needs to be updated. Existing application programs continue to function without modification in most cases.

Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4

Figure 1.4
Internal storage format
for a STUDENT record,
based on the database
catalog in Figure 1.3.

This separation between programs and data is achieved through **data abstraction**. Data abstraction hides unnecessary implementation details from users and provides a simplified conceptual view of the database. Users interact with logical concepts such as tables, attributes, and relationships without needing to understand how the data is physically stored on disk.

Thus, data abstraction improves flexibility, simplifies database usage, and makes application development easier.

3. Support for Multiple Views of Data

A database is generally used by many users belonging to different departments. Each user has different information requirements and does not need access to the entire database. Therefore, a DBMS provides the facility of **multiple views**.

A **view** is a customized representation of the database that contains only the information required by a particular user or application. A view may consist of selected rows, selected columns, or even virtual data derived from existing tables.

For example:

- A **student** may only need to view personal details and examination results.
- A **faculty member** may require student attendance and marks.
- The **accounts department** may access only fee-related information.
- The **library staff** may require details related to book borrowing.

Each user interacts only with the relevant portion of the database, improving security, simplicity, and efficiency.

4. Sharing of Data and Multiuser Transaction Processing

One of the most powerful features of a DBMS is its ability to allow multiple users to access the database simultaneously. Since a single database is shared among different users and applications, the DBMS must ensure that concurrent operations do not lead to incorrect or inconsistent data.

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

(a)

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

(b)

Figure 1.5

Two views derived from the database in Figure 1.2. (a) The TRANSCRIPT view. (b) The COURSE_PREREQUISITES view.

To achieve this, the DBMS uses **concurrency control** techniques. Concurrency control ensures that when multiple users attempt to read or update the same data simultaneously, the operations are executed in a controlled and consistent manner.

For example, if two airline reservation agents try to book the same seat at the same time, the DBMS ensures that only one booking is successful, preventing duplicate seat allocation.

The operations performed by users are grouped into **transactions**. A transaction is a sequence of one or more database operations that together perform a single logical task. The DBMS guarantees important transaction properties such as:

- **Atomicity:** Either all operations of a transaction are completed successfully or none of them are performed.
- **Isolation:** Each transaction executes independently without interference from other concurrent transactions.

These properties ensure data consistency, reliability, and correctness in multiuser environments.

The database approach provides several advantages over the traditional file processing system. By maintaining a centralized database, separating programs from data, supporting multiple user views, and allowing secure concurrent access, a DBMS improves data consistency, reduces redundancy, enhances flexibility, and simplifies database management. These characteristics make the database approach suitable for modern applications such as banking systems, hospitals, universities, airline reservation systems, and e-commerce platforms.

1.4 Actors on the Scene

Introduction

A database system is rarely used by a single person. In small personal databases, such as a personal contact list or address book, one individual usually creates, manages, and uses the database. Since the database is not shared with others, there is no need for multiple roles or responsibilities.

However, in large organizations such as universities, banks, hospitals, airlines, and multinational companies, a database is shared among hundreds or even thousands of users. Such large databases require different people to perform specialized tasks such as designing the database, maintaining it, developing applications, and using the stored data. These individuals are collectively known as **Actors on the Scene** because they interact with the database in their day-to-day activities.

The major actors involved in a database environment are:

- Database Administrator (DBA)
- Database Designer
- End Users
- System Analysts and Application Programmers

1. Database Administrator (DBA)

The **Database Administrator (DBA)** is the person responsible for the overall management and control of the database system. Just as every organization requires a

manager to supervise its resources, every database environment requires a DBA to manage both the database and the Database Management System (DBMS). The DBA ensures that the database remains secure, efficient, reliable, and available to authorized users.

The responsibilities of a DBA include authorizing user access, monitoring database usage, coordinating database activities, and managing the required hardware and software resources. The DBA is also responsible for handling security issues, preventing unauthorized access, maintaining backup and recovery mechanisms, and resolving performance-related problems such as slow response time. In large organizations, the DBA is usually supported by a team of database professionals who assist in performing these tasks.

Responsibilities of a DBA

- Authorizes users to access the database.
- Monitors and controls database usage.
- Maintains database security.
- Performs backup and recovery operations.
- Acquires and manages database software and hardware.
- Resolves performance and system-related issues.

2. Database Designers

A **Database Designer** is responsible for designing the structure of the database before it is implemented. The designer identifies what data needs to be stored and determines the most suitable way to organize and represent that data. A well-designed database improves efficiency, minimizes redundancy, and supports the needs of all users.

To achieve this, database designers interact with different groups of users to understand their requirements. Based on these requirements, they prepare different views of the database and integrate them into a single database design. The final design should satisfy the information needs of all users while maintaining consistency and efficiency. In many organizations, database designers work under the supervision of the DBA and may perform additional responsibilities after the database has been implemented.

Responsibilities of Database Designers

- Identify the data to be stored.
- Design the database structure.
- Communicate with users to gather requirements.
- Create user-specific database views.
- Integrate all user requirements into a single database design.

3. End Users

End Users are the people who directly use the database in their daily work. The primary purpose of developing a database is to provide useful information to these users. They access the database to retrieve information, update records, and generate reports according to their job requirements.

Depending on their level of knowledge and the way they interact with the database, end users are classified into four categories.

a) Casual End Users

Casual end users access the database only occasionally. They usually require different information each time they use the system and therefore rely on flexible query interfaces to retrieve data. These users are generally managers or executives who use the database mainly for decision-making purposes rather than routine operations.

Examples:

- Department Managers
- Company Executives
- Administrative Officers

b) Naive or Parametric End Users

Naive or parametric end users perform routine and repetitive database operations using predefined application programs known as **canned transactions**. They do not require knowledge of SQL or database internals because the application interface performs all necessary database operations. This category forms the largest group of database users.

Examples:

- Bank customers and bank tellers

- Airline, hotel, and railway reservation agents
- Courier and shipping employees
- Social media users

c) Sophisticated End Users

Sophisticated end users possess a good understanding of the DBMS and frequently use advanced database features. They often develop their own queries and applications to satisfy complex information requirements. These users are generally technical professionals who work with large amounts of data.

Examples:

- Engineers
- Scientists
- Business Analysts
- Data Analysts

d) Standalone Users

Standalone users maintain their own personal databases using ready-made software packages that provide graphical or menu-driven interfaces. These users do not require programming knowledge and generally become proficient in using a particular application for managing personal information.

Examples:

- Personal finance software users
- Home inventory management users
- Personal contact management users

4. System Analysts and Application Programmers (Software Engineers)

System Analysts and Application Programmers play an important role in developing database applications that meet user requirements. They act as a bridge between end users and the database system.

A **System Analyst** studies the requirements of end users, particularly naive and parametric users, and prepares detailed specifications for the required database

applications. These specifications describe how the application should function and what operations it should perform.

An **Application Programmer**, also known as a **Software Engineer**, converts these specifications into working programs. The programmer develops the application, tests it thoroughly, removes errors, prepares documentation, and performs future maintenance whenever required. To carry out these tasks effectively, software engineers must have a good understanding of the capabilities provided by the DBMS.

Responsibilities

System Analyst

- Studies user requirements.
- Prepares system specifications.
- Designs standard database transactions.

Application Programmer

- Develops database applications.
- Implements database transactions.
- Tests and debugs programs.
- Documents and maintains the software.

In a large database environment, different people perform specialized roles to ensure the efficient functioning of the database system. The **Database Administrator (DBA)** manages and secures the database, the **Database Designer** creates the database structure, **End Users** interact with the database to perform daily operations, and **System Analysts and Application Programmers** develop applications that enable users to access and manipulate data effectively. Together, these actors ensure that the database system operates efficiently, securely, and according to organizational requirements.

1.5 Workers behind the Scene

In addition to those who design, use, and administer a database, others are associated with the design, development, and operation of the DBMS software and system environment. These persons are typically not interested in the database content itself. We call them the workers behind the scene, and they include the following categories:

- DBMS system designers and implementers design and implement the DBMS modules and interfaces as a software package. A DBMS is a very complex software system that consists of many components, or modules, including modules for implementing the catalog, query language processing, interface processing, accessing and buffering data, controlling concurrency, and handling data recovery and security. The DBMS must interface with other system software, such as the operating system and compilers for various programming languages.
- Tool developers design and implement tools—the software packages that facilitate database modeling and design, database system design, and improved performance. Tools are optional packages that are often purchased separately. They include packages for database design, performance monitoring, natural language or graphical interfaces, prototyping, simulation, and test data generation. In many cases, independent software vendors develop and market these tools.
- Operators and maintenance personnel (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system. Although these categories of workers behind the scene are instrumental in making the database system available to end users, they typically do not use the database contents for their own purposes.

1.6 Advantages of Using the DBMS Approach (Part 1)

Introduction

A **Database Management System (DBMS)** provides several advantages over the traditional file processing system. In addition to the four fundamental characteristics of the database approach, a DBMS offers many advanced capabilities that improve data management, security, efficiency, and reliability. These capabilities help organizations store and manage large volumes of data while ensuring that the data remains consistent, secure, and easily accessible. The Database Administrator (DBA) utilizes these capabilities to design, maintain, and administer large multi-user databases effectively.

The major advantages of using the DBMS approach are:

1. **Controlling Redundancy**
2. **Restricting Unauthorized Access**

3. Providing Persistent Storage for Program Objects
4. Providing Storage Structures and Search Techniques for Efficient Query Processing
5. Providing Backup and Recovery
6. Providing Multiple User Interfaces
7. Representing Complex Relationships among Data
8. Enforcing Integrity Constraints
9. Permitting Inferencing and Actions using Rules and Triggers
10. Additional Organizational Benefits

1.6.1 Controlling Redundancy

Introduction

One of the most significant advantages of a DBMS is its ability to **control data redundancy**. Data redundancy refers to the unnecessary duplication of the same data at multiple locations. In the traditional file processing system, each department maintains its own separate files to meet its specific requirements. As a result, the same information is stored repeatedly in different files, leading to duplication, inconsistency, and wastage of storage space.

For example, in a university environment, the **Accounts Department** maintains student records for fee collection, while the **Registration Department** maintains student records for course registration and grades. Since both departments store student information independently, the same student's details are duplicated in multiple files. Whenever a new student joins the university, the information has to be entered separately into each departmental file. This increases the amount of work and creates the possibility of errors.

Figure 1.6

Redundant storage of Student_name and Course_name in GRADE_REPORT.
 (a) Consistent data.
 (b) Inconsistent record.

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

(a)

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Brown	112	MATH2410	B

(b)

Problems Caused by Redundancy

When the same information is stored multiple times, several problems arise.

1. Duplication of Effort

Every update must be performed repeatedly in all files containing the same information. This increases the workload and consumes additional time.

Example:

If a new student joins the university, the student's details must be entered into both the registration database and the accounts database.

2. Wastage of Storage Space

Repeated storage of identical information unnecessarily occupies disk space. This becomes a serious problem in large organizations where millions of records are stored.

3. Data Inconsistency

The most serious problem caused by redundancy is **data inconsistency**.

Data inconsistency occurs when different copies of the same data contain different values because some files are updated while others are not.

For example,

Suppose the actual birth date of a student is

29-Jan-1988

If one department stores

19-Jan-1988

while another department stores

29-Jan-1988

both files contain conflicting information about the same student. This situation is known as **data inconsistency**. Such inconsistencies reduce the reliability of the database and may lead to incorrect decision-making.

How DBMS Controls Redundancy

The DBMS minimizes redundancy by maintaining a **single integrated database** instead of separate departmental files. During database design, the requirements of different user groups are combined into one unified database.

Ideally, every logical data item, such as a student's name, registration number, or birth date, is stored only once. All users access this common copy whenever required. This approach eliminates duplicate storage, ensures consistency, and saves storage space.

Data Normalization

The process of organizing the database so that each logical data item is stored only once is called **Data Normalization**.

Normalization helps in:

- Eliminating unnecessary duplication of data.
- Maintaining consistency.
- Saving storage space.
- Simplifying database maintenance.

Thus, normalization is one of the primary techniques used by DBMS to control redundancy.

Controlled Redundancy (Denormalization)

Although redundancy should generally be minimized, there are situations where storing duplicate information intentionally can improve system performance.

This technique is known as **Controlled Redundancy** or **Denormalization**.

For example, the **GRADE_REPORT** table may store the student's name and course number along with the student number and grade. Since these values are frequently required together, storing them in the same table reduces the need to search multiple tables during query execution, thereby improving query performance.

However, whenever redundancy is introduced intentionally, the DBMS must ensure that all duplicate copies remain consistent by automatically checking their correctness during updates.

Advantages of Controlling Redundancy

- Eliminates duplicate data.
- Saves storage space.
- Prevents data inconsistency.
- Reduces maintenance effort.
- Improves data accuracy.
- Increases overall database reliability.

1.6.2 Restricting Unauthorized Access

Introduction

In a multi-user database environment, not every user should be allowed to access all the information stored in the database. Certain data, such as employee salaries, examination results before publication, medical records, or financial transactions, is confidential and should only be available to authorized individuals.

Therefore, a DBMS provides a **security and authorization mechanism** that restricts database access according to user roles and responsibilities.

Need for Authorization

Different users require different levels of access.

For example,

- A student can view only his or her own academic records.
- A faculty member can update students' marks.
- The accounts department can access fee-related information.
- The Database Administrator has complete control over the database.

Providing the same privileges to every user would create serious security risks.

Security Mechanism in DBMS

The DBMS allows the Database Administrator (DBA) to create individual user accounts and assign permissions according to organizational requirements.

The authorization system controls:

- Who can access the database.
- Which tables can be accessed.
- What operations can be performed.
- Which users can modify or delete data.

Typically, users authenticate themselves using **user IDs and passwords** before accessing the database. After successful authentication, the DBMS verifies the permissions assigned to that user and allows only authorized operations.

Types of Access Permissions

A DBMS may provide permissions such as:

- Read (Retrieve data)
- Insert (Add new records)
- Update (Modify existing records)
- Delete (Remove records)
- Create or Alter database objects

Different users receive different combinations of these privileges depending on their responsibilities.

Example

In a university database,

- Students can only **view** their marks.
- Faculty members can **enter and modify** marks.
- The Accounts Office can access **fee records**.
- Only the **DBA** can create new user accounts or change database permissions.

This ensures that sensitive information remains protected while allowing authorized users to perform their assigned tasks.

Advantages of Authorization

- Protects confidential information.
- Prevents unauthorized modifications.
- Improves database security.
- Maintains data integrity.
- Enables controlled access for different users.
-

1.6.3 Providing Persistent Storage for Program Objects

Introduction

In conventional programming languages such as C++ and Java, program variables and objects exist only while the program is running. Once the program terminates, these objects are removed from memory unless they are explicitly stored in permanent files.

A DBMS solves this problem by providing **persistent storage**, allowing program objects to be stored permanently inside the database. Such objects continue to exist even after program execution has ended and can be retrieved whenever required by another program.

Object-oriented database systems support this feature by automatically storing and retrieving complex program objects without requiring programmers to manually convert them into file formats. This eliminates the **impedance mismatch** between programming language data structures and database storage formats, making application development simpler and more efficient.

1.6.4 Providing Storage Structures and Search Techniques for Efficient Query Processing

Introduction

A DBMS is designed to process queries and updates efficiently, even when dealing with very large databases. Since database records are usually stored on secondary storage devices such as hard disks, searching for required information can be time-consuming if no optimization techniques are used.

To improve performance, the DBMS provides specialized **storage structures** and **search techniques** that reduce the time required to retrieve records.

The DBMS makes use of **indexes**, which are auxiliary data structures built using tree-based or hash-based techniques. Instead of scanning the entire database, the DBMS searches the index first to locate the required records quickly.

In addition, the DBMS maintains **buffers (cache memory)** that temporarily store frequently accessed data in main memory. This reduces repeated disk accesses and significantly improves query performance. The DBMS also contains a **query processor** and **query optimizer**, which analyze each query and determine the most efficient execution plan for retrieving the required data.

1.6.5 Providing Backup and Recovery

Introduction

Data is one of the most valuable assets of any organization. However, data may be lost due to hardware failures, software crashes, power outages, accidental deletion, malicious attacks, or natural disasters. Such failures can interrupt database operations and may lead to permanent loss of valuable information if proper recovery mechanisms are not available. Therefore, a Database Management System (DBMS) provides a **Backup and Recovery** subsystem to protect the database against unexpected failures.

The recovery subsystem continuously monitors database transactions and maintains sufficient information to restore the database whenever a failure occurs. If a system failure happens while a transaction is being executed, the DBMS restores the database to the last consistent state before the failure. This ensures that incomplete transactions do not corrupt the database.

Apart from transaction recovery, the DBMS also supports periodic **database backups**. A backup is a duplicate copy of the database stored in a secure location. In the event of hardware failure or complete disk damage, the database can be reconstructed using these backup copies, thereby preventing permanent data loss.

Advantages

- Protects data against hardware and software failures.
- Prevents permanent loss of information.
- Restores the database to a consistent state.
- Ensures business continuity.
- Supports disaster recovery.

1.6.6 Providing Multiple User Interfaces

Introduction

A database is used by different categories of users, each possessing different levels of technical knowledge. While programmers may prefer programming language interfaces, casual users may require query languages, and beginners may prefer graphical interfaces. Therefore, a DBMS provides **multiple user interfaces** so that every category of user can interact with the database conveniently.

The interface selected depends upon the user's experience and the nature of the task being performed. By offering multiple interfaces, the DBMS becomes easier to use, reduces the learning curve, and improves overall productivity.

Common interfaces provided by a DBMS include:

- Query language interfaces (SQL)
- Programming language interfaces

- Menu-driven interfaces
- Forms-based interfaces
- Graphical User Interfaces (GUI)
- Web-based interfaces
- Mobile application interfaces
- Natural language interfaces

Modern DBMS software extensively uses **Graphical User Interfaces (GUI)** and web-enabled applications because they provide a simple and interactive environment for users to access the database.

Advantages

- Makes database access easier.
- Supports different categories of users.
- Improves user productivity.
- Reduces training requirements.
- Enhances user experience.

1.6.7 Representing Complex Relationships Among Data

Introduction

Real-world applications contain a large number of entities that are interconnected in different ways. A good DBMS must be capable of representing these **complex relationships** efficiently. Unlike traditional file systems, which store data independently, a DBMS allows related information to be connected through relationships, enabling meaningful retrieval and management of data.

For example, in a university database:

- A single student can enroll in many course sections.
- Each course section may contain many students.
- Every section belongs to one course.

- Each course may have one or more prerequisite courses.

The DBMS stores these relationships so that users can easily retrieve related information whenever required. It also allows new relationships to be created as organizational requirements evolve. Efficient representation of relationships reduces redundancy and simplifies data retrieval.

Advantages

- Represents real-world relationships effectively.
- Simplifies retrieval of related information.
- Supports complex applications.
- Reduces duplication of data.
- Improves database consistency.

1.6.8 Enforcing Integrity Constraints

Introduction

A database should always contain **accurate, valid, and consistent** information. To ensure this, a DBMS provides mechanisms for defining and enforcing **Integrity Constraints**. Integrity constraints are rules that restrict the values stored in the database and ensure that the data remains correct throughout its lifetime.

The simplest integrity constraint specifies the **data type** of an attribute. For example, the attribute **Class** should contain only integer values, while **Name** should contain only alphabetic characters. Additional constraints may restrict the range of values. For instance, the class value may be allowed only between **1 and 5**.

The DBMS also enforces **referential integrity**, which ensures that relationships between tables remain valid. For example, every record in the **SECTION** table must correspond to an existing course in the **COURSE** table. Similarly, **key constraints** guarantee that attributes such as **Course_Number** contain unique values. These constraints maintain consistency and prevent invalid data from entering the database.

Although integrity constraints can prevent many errors, they cannot detect logically incorrect but valid values. For example, entering grade **C** instead of the correct grade

A cannot be identified automatically because C is still a valid grade. However, entering grade Z will be rejected since it violates the defined data type.

Advantages

- Ensures data accuracy.
- Maintains consistency.
- Prevents invalid data entry.
- Preserves relationships among tables.
- Improves database reliability.

1.6.9 Permitting Inferencing and Actions Using Rules and Triggers

Introduction

Modern DBMSs provide advanced mechanisms that allow the database to automatically derive new information and perform actions whenever specified conditions are satisfied. These mechanisms are implemented using **Rules**, **Triggers**, and **Stored Procedures**.

A **Rule** enables the database to infer new information from existing facts. For example, a university database may automatically determine whether a student should be placed on probation based on predefined academic regulations.

A **Trigger** is a special procedure that is automatically executed whenever a specific database event occurs, such as inserting, updating, or deleting a record. For example, whenever a student's marks are updated, a trigger may automatically recalculate the student's GPA or update another related table.

A **Stored Procedure** is a predefined sequence of SQL statements stored inside the database. These procedures execute automatically whenever certain conditions are satisfied, reducing repetitive programming and improving efficiency.

Advantages

- Automates routine operations.
- Reduces manual intervention.
- Maintains consistency.

- Improves efficiency.
- Supports intelligent database applications.

1.6.10 Additional Implications of Using the Database Approach

Apart from the major advantages discussed above, the database approach provides several additional benefits that improve organizational efficiency and simplify database management.

1. Potential for Enforcing Standards

Since all organizational data is stored in a centralized database, the Database Administrator (DBA) can define and enforce uniform standards for naming conventions, data formats, terminology, report layouts, and coding schemes. Standardization improves communication, simplifies maintenance, and ensures consistency across different departments.

2. Reduced Application Development Time

Developing a new application becomes much easier because the database already exists. Developers only need to create new programs that utilize the existing database rather than building new files from scratch. Consequently, application development using a DBMS requires significantly less time than traditional file processing systems.

3. Flexibility

Organizational requirements change continuously. A DBMS allows modifications such as adding new attributes, creating new tables, or introducing additional user groups without significantly affecting existing applications. This flexibility enables the database to evolve with changing business needs.

4. Availability of Up-to-Date Information

Since all users access the same centralized database, any update performed by one user becomes immediately available to all other authorized users. This ensures that everyone works with the most recent information, which is especially important in banking, airline reservation, and online shopping systems.

5. Economies of Scale

The database approach allows organizations to consolidate data, software, and hardware resources into a centralized system. Instead of each department maintaining separate databases and computing resources, all departments share a common infrastructure. This reduces operational costs, minimizes duplication of resources, and improves overall system efficiency.

The DBMS approach provides numerous advantages over the traditional file processing system. It controls redundancy, protects data through security mechanisms, supports efficient storage and query processing, provides backup and recovery facilities, represents complex relationships, enforces integrity constraints, and automates database operations using rules and triggers. Additionally, it offers organizational benefits such as standardization, flexibility, reduced application development time, availability of up-to-date information, and cost savings. These advantages make the DBMS an essential component of modern information systems and enable organizations to manage large volumes of data efficiently and securely.

1.7 A Brief History of Database Applications

Introduction

The evolution of Database Management Systems (DBMSs) has been driven by the continuously changing requirements of organizations and applications. As businesses and technologies advanced, the amount, complexity, and variety of data also increased. Consequently, database systems evolved from simple file-based systems to modern relational, object-oriented, XML-based, and NoSQL databases. Each new generation of database systems was developed to overcome the limitations of its predecessor and to support emerging application requirements. Understanding this evolution helps us appreciate why different database models exist today and how they continue to adapt to modern computing environments.

The historical development of database applications can be divided into the following stages:

1. Early Database Applications using Hierarchical and Network Systems
2. Relational Databases
3. Object-Oriented Databases
4. XML and Web-based Databases

5. Database Extensions for Specialized Applications

6. Big Data and NoSQL Databases

1.7.1 Early Database Applications Using Hierarchical and Network Systems

Introduction

The earliest database systems were developed during the **1960s, 1970s, and early 1980s** to manage large collections of records in organizations such as banks, universities, hospitals, and business enterprises. These systems handled data such as student records, employee information, patient records, banking transactions, and inventory details. Since organizations stored thousands of similar records, hierarchical and network database models became the primary approaches for organizing this information.

Although these early systems were successful in storing large amounts of data, they suffered from several limitations.

One of the major drawbacks was the **mixing of conceptual relationships with physical storage**. The physical location of records on the disk was closely tied to how the data was logically related. For example, a student's grade records were often stored physically next to the student's record. While this arrangement made certain queries very fast, it greatly reduced the flexibility of the system. Whenever new types of queries were introduced, the existing storage organization often became inefficient, and reorganizing the database was both difficult and time-consuming.

Another limitation was the lack of **data abstraction** and **program-data independence**. Since application programs depended heavily on the physical organization of data, even small changes in the database structure required modifications to the application programs. These systems also supported only programming language interfaces, meaning that every new query or transaction required writing, testing, and debugging a separate program. As a result, application development was slow, expensive, and inflexible.

The major early database systems included:

- Hierarchical Database Systems
- Network Database Systems
- Inverted File Systems

Despite their limitations, these systems laid the foundation for modern database technology.

1.7.2 Providing Data Abstraction and Application Flexibility with Relational Databases

Introduction

To overcome the shortcomings of early database systems, the **Relational Database Model** was proposed. The primary objective of relational databases was to separate the physical storage of data from its logical representation, thereby improving **data abstraction** and **program-data independence**. Instead of depending on physical storage locations, data was represented in the form of **relations (tables)** consisting of rows and columns.

A major contribution of the relational model was the introduction of **high-level query languages**, allowing users to retrieve and manipulate data without writing complex application programs. This significantly reduced the time required to develop new queries and applications.

Initially, relational database systems were slower than earlier database systems because they did not use physical storage pointers to locate related records. However, continuous improvements in indexing techniques, storage structures, query optimization, and processing algorithms gradually enhanced their performance. As a result, relational databases became the most widely used database systems for traditional business applications. Today, relational databases are used in almost every sector, ranging from personal computers to large enterprise servers.

Advantages of Relational Databases

- Better data abstraction.
- Program-data independence.
- Flexible database design.
- High-level query language support.
- Easier database maintenance.

1.7.3 Object-Oriented Applications and the Need for More Complex Databases

Introduction

During the **1980s**, object-oriented programming languages such as **C++** and **Java** became increasingly popular. These languages introduced concepts such as objects, classes, inheritance, and encapsulation. Traditional relational databases were not well suited to storing these complex objects, leading to the development of **Object-Oriented Database Systems (OODBs)**.

Object-oriented databases allowed complex objects to be stored directly in the database while preserving object-oriented concepts such as **abstract data types, encapsulation, inheritance, and object identity**. This made them particularly suitable for applications involving engineering design, manufacturing, multimedia systems, and scientific research.

Although OODBs offered several advantages, their adoption remained limited because of their complexity and the absence of a common industry standard. Instead of replacing relational databases, many object-oriented concepts were integrated into relational systems, resulting in **Object-Relational Database Management Systems (ORDBMSs)**, which combined the strengths of both models.

1.7.4 Interchanging Data on the Web for E-Commerce Using XML

Introduction

The rapid growth of the **World Wide Web (WWW)** during the **1990s** transformed the way information was shared across the Internet. Static web pages created using **HyperText Markup Language (HTML)** allowed users to access information through web browsers. However, the emergence of **electronic commerce (e-commerce)** created a need for dynamic web pages that displayed continuously changing information such as flight schedules, product prices, stock availability, and customer records.

To facilitate the exchange of structured information between databases and web applications, the **eXtensible Markup Language (XML)** was introduced. XML provided a standard format for storing and exchanging data across different database systems and web applications. By combining concepts from document management

systems and database systems, XML became an important technology for web-based data interchange.

Applications

- E-commerce websites
- Online booking systems
- Product catalogs
- Data exchange between organizations

1.7.5 Extending Database Capabilities for New Applications

Introduction

As technology advanced, databases were required to support applications that handled much more complex data than simple numerical or textual information. Scientific research, multimedia applications, geographic information systems, and data mining required new storage techniques, specialized data types, and advanced query capabilities.

Some important application areas include:

- Scientific databases
- Image databases
- Video databases
- Data mining systems
- Spatial databases
- Time-series databases

Traditional relational databases were not sufficiently flexible to support these applications because they mainly handled simple tables and basic data types. These new applications required:

- More complex data structures.
- New data types.
- Specialized query operations.

- Advanced storage and indexing techniques.

To meet these requirements, DBMS developers extended relational systems by incorporating object-oriented features and optional modules designed specifically for specialized applications.

1.7.6 Emergence of Big Data Storage Systems and NoSQL Databases

Introduction

During the **twenty-first century**, the rapid growth of social media platforms, cloud computing, e-commerce companies, and web search engines generated enormous volumes of data, commonly referred to as **Big Data**. Managing such massive datasets using only traditional relational databases became increasingly difficult because of scalability and performance challenges.

To address these challenges, new database systems known as **NoSQL databases** were introduced. The term **NoSQL** stands for "**Not Only SQL**," indicating that modern applications may use both relational (SQL) databases and non-relational databases depending on their requirements. NoSQL systems are particularly suitable for storing unstructured and semi-structured data such as social media posts, tweets, documents, and cloud-based information. They provide fast storage, retrieval, scalability, and fault tolerance for very large datasets.

Today, organizations often combine SQL and NoSQL databases to achieve the best performance for different types of applications.

The history of database applications reflects the continuous evolution of technology to meet changing business and computing requirements. Early hierarchical and network systems introduced organized data storage but lacked flexibility. Relational databases revolutionized database management by providing data abstraction, program-data independence, and high-level query languages. Object-oriented databases extended support for complex objects, while XML enabled efficient web-based data exchange. As applications became increasingly specialized, database systems incorporated new features to support multimedia, scientific, spatial, and data mining applications. Finally, the emergence of Big Data and NoSQL databases addressed the challenges of managing massive volumes of structured, semi-structured, and unstructured data. Today, modern database systems combine the

strengths of multiple database models to support diverse real-world applications efficiently.

1.8 When Not to Use a DBMS

Introduction

A Database Management System (DBMS) provides numerous advantages such as reduced data redundancy, improved security, data integrity, backup and recovery, concurrent access, and efficient data management. Because of these benefits, DBMSs are widely used in modern organizations. However, a DBMS is **not the best solution for every application**. In certain situations, the additional features offered by a DBMS introduce unnecessary complexity, cost, and performance overhead. In such cases, traditional file processing systems or customized database applications may be more appropriate.

The decision to use a DBMS should therefore be based on the requirements of the application. If an application is simple, has fixed requirements, or operates under strict resource constraints, using a full-fledged DBMS may not be economical or efficient.

Overhead Costs of Using a DBMS

Although a DBMS offers many advanced features, these features require additional resources. The overhead associated with a DBMS mainly arises from the following factors.

1. High Initial Investment

Implementing a DBMS requires a considerable initial investment. Organizations must purchase suitable hardware, DBMS software licenses, storage devices, and networking equipment. In addition, users, administrators, and developers require proper training to use and manage the system effectively. Consequently, the initial cost of deploying a DBMS is much higher than that of a traditional file processing system.

2. Generality of the DBMS

A DBMS is designed to support a wide variety of applications and users. It provides many general-purpose features such as query processing, transaction management,

concurrency control, security, and recovery mechanisms. While these features increase flexibility, they also introduce additional processing overhead. For very simple applications, many of these capabilities may never be used, making the DBMS unnecessarily complex.

3. Overhead of Advanced Features

A DBMS continuously performs several internal operations to ensure correct and reliable database management. These include:

- Security and authorization
- Concurrency control
- Backup and recovery
- Integrity constraint enforcement
- Transaction management

Although these features improve reliability and data consistency, they consume additional CPU time, memory, and storage resources, thereby increasing the overall system overhead.

Situations Where a DBMS May Not Be Suitable

A DBMS is not always the most appropriate choice. In the following situations, a customized file processing system may provide better performance and lower cost.

1. Simple and Well-Defined Applications

If an application is very small, performs only a limited number of operations, and is not expected to change in the future, implementing a DBMS may be unnecessary. A simple file-based application can perform the required tasks efficiently without the additional complexity of database management.

Example:

A small shop maintaining a fixed list of products using simple data files.

2. Real-Time Applications with Stringent Performance Requirements

Some applications require extremely fast response times. In such systems, even the small processing overhead introduced by a DBMS may be unacceptable. Since the

DBMS performs security checks, concurrency control, recovery, and transaction processing, these additional operations may reduce performance.

For applications where every millisecond is important, specially designed database software or customized file systems are often preferred.

Example:

Real-time industrial control systems or communication switching systems.

3. Embedded Systems with Limited Resources

Embedded systems usually have limited memory, storage capacity, and processing power. Installing a general-purpose DBMS in such systems may not be practical because it requires additional storage and computational resources.

Instead, lightweight file management techniques or specialized embedded database solutions are generally used.

Example:

- Smart home devices
- Embedded medical equipment
- Automotive control systems

4. Applications Without Multiple Users

If only one user accesses the data and there is no requirement for sharing information among multiple users, many DBMS features become unnecessary. Facilities such as concurrency control, transaction management, and multi-user security provide little benefit in such situations. Therefore, a simple file processing system may be sufficient.

Example:

A personal contact manager used by a single individual.

Applications That Commonly Avoid General-Purpose DBMS

Certain specialized industries prefer customized database software because their applications require highly optimized storage structures and operations that are not efficiently supported by general-purpose DBMSs.

1. Computer-Aided Design (CAD) Systems

CAD applications used by mechanical and civil engineers handle engineering drawings, blueprints, and three-dimensional models. These systems often use proprietary file management software specifically designed for efficient manipulation of graphical objects rather than relying on a general-purpose DBMS.

2. Communication and Switching Systems

Telecommunication companies require extremely fast processing of call routing information. Early communication and switching systems therefore used specialized database software optimized for hierarchical data structures to achieve very high performance.

3. Geographic Information Systems (GIS)

GIS applications manage geographical maps, satellite images, contours, roads, rivers, polygons, and spatial coordinates. Since these applications involve specialized spatial operations, they often implement customized storage and indexing mechanisms instead of using a general-purpose DBMS.

Advantages of Using Customized File Systems in Such Applications

In applications where a DBMS is unnecessary, customized systems provide several benefits:

- Lower implementation cost.
- Faster execution for specialized tasks.
- Reduced memory and storage requirements.
- Simpler system design.
- Better performance for fixed and specialized applications.

Although a Database Management System provides powerful features for managing data, it is not universally suitable for every application. The cost of hardware, software, training, and the processing overhead associated with security, recovery, concurrency control, and transaction management may outweigh its benefits in small or highly specialized systems. Therefore, applications that are simple, have fixed requirements, operate under strict real-time constraints, use limited hardware resources, or do not require multi-user access may be better implemented using customized file processing systems. Specialized domains such as **CAD**,

communication systems, and **GIS** often rely on proprietary data management solutions because they provide higher efficiency for their specific requirements.

2.1 Data Models, Schemas, and Instances

Introduction

One of the most important characteristics of the database approach is **data abstraction**. Data abstraction refers to the process of hiding unnecessary details of data storage and organization while emphasizing the essential characteristics of the data. Instead of exposing users to complex storage structures, a database system presents information at an appropriate level of detail depending on the user's requirements. This makes databases easier to understand, design, and use.

To achieve data abstraction, a DBMS uses a **Data Model**. A data model provides a collection of concepts that describe the structure of a database, including the data types, relationships, constraints, and operations that can be performed on the data. In addition to defining the static structure of a database, modern data models also support the specification of user-defined operations and behaviors, enabling database objects to perform specific functions.

2.1.1 Data Model

Definition

A **Data Model** is a collection of concepts used to describe the structure of a database. It specifies how data is organized, how different data items are related, the constraints that apply to the data, and the operations that can be performed for retrieving and updating information. It forms the foundation for database design and provides the necessary abstraction between users and the physical storage of data.

Besides defining the structure of the database, many modern data models also allow the specification of **behavior** through user-defined operations. For example, a database may include an operation such as **COMPUTE_GPA**, which calculates the grade point average of a student. While operations like insert, delete, update, and retrieve are generic operations supported by almost every DBMS, user-defined operations allow developers to implement application-specific functionality directly within the database. These concepts are widely used in object-oriented and object-relational database systems.

Components of a Data Model

A data model generally consists of the following components:

- **Data Structure:** Defines how data is organized.
- **Relationships:** Specify the associations among different data items.
- **Constraints:** Define rules that ensure data validity and consistency.
- **Operations:** Provide methods for inserting, deleting, updating, and retrieving data.

Categories of Data Models

Depending on the level of abstraction they provide, data models are classified into the following categories:

1. Conceptual (High-Level) Data Models
2. Representational (Implementation) Data Models
3. Physical (Low-Level) Data Models
4. Self-Describing Data Models

1. Conceptual (High-Level) Data Models

Conceptual data models provide a high level of abstraction and describe the database from the user's point of view. These models focus on real-world objects and their relationships rather than on how the data is physically stored in the computer. Because they closely resemble the way users think about data, conceptual models are primarily used during the database design phase.

The three basic concepts used in conceptual data models are:

Entity

An **Entity** is a real-world object, person, place, event, or concept about which data is stored in the database.

Examples:

- Student
- Employee

- Course
- Department

Attribute

An **Attribute** is a property or characteristic that describes an entity.

Examples:

For a **Student** entity:

- Student_ID
- Name
- Age
- Major

Relationship

A **Relationship** represents the association between two or more entities.

Example:

A **Student** enrolls in a **Course**.

This relationship connects the Student entity with the Course entity.

Example

The **Entity-Relationship (ER) Model** is the most widely used conceptual data model.

2. Representational (Implementation) Data Models

Representational data models occupy the middle level between conceptual and physical models. They are easier for end users to understand than physical models while still being directly implementable by database systems. These models hide most physical storage details but represent data in a structured format suitable for implementation.

The most commonly used representational models include:

- Relational Data Model
- Hierarchical Data Model

- Network Data Model

Since these models represent information using records, they are also called **Record-Based Data Models**.

Among these, the **Relational Data Model** is the most popular and forms the basis of modern relational database management systems (RDBMS).

3. Physical (Low-Level) Data Models

Physical data models describe **how data is actually stored inside the computer**. These models specify the internal organization of database files, record formats, storage locations, indexing methods, hashing techniques, and access paths. Physical models are mainly intended for database administrators and system developers rather than end users.

Important concepts include:

- Record Format
- Record Ordering
- Indexing
- Hashing
- Access Paths

An **Access Path** is a search structure that enables efficient retrieval of database records. Indexes are common examples of access paths.

4. Self-Describing Data Models

In traditional databases, the schema (description of data) is stored separately from the actual data. However, self-describing data models combine both the data description and the data values within the same structure. This makes the data self-contained and easier to exchange between different systems.

Examples include:

- XML
- Key-Value Stores
- NoSQL Databases

Comparison of Data Models

Conceptual Data Model	Representational Data Model	Physical Data Model
High level	Medium level	Low level
User-oriented	DBMS-oriented	Hardware-oriented
Uses entities and relationships	Uses tables or records	Uses files and storage structures
Easy to understand	Moderate complexity	Difficult for end users
Used during database design	Used for implementation	Used for physical storage

2.1.2 Database Schema, Instance, and Database State

Database Schema

Definition

A **Database Schema** is the overall logical design or blueprint of a database. It specifies the structure of the database, including tables, attributes, relationships, and constraints. The schema is created during database design and normally remains unchanged unless the application requirements change.

A schema describes **what information will be stored**, but it does **not** contain the actual data.

Figure 2.1

Schema diagram for the database in Figure 1.2.

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

For example, in a university database:

STUDENT

- Student_ID
- Name
- Class
- Major

This definition represents the schema of the STUDENT table.

Schema Diagram

A **Schema Diagram** is a graphical representation of the database schema. It shows the names of tables (relations), their attributes, and some structural constraints, but it does not display the actual data stored in the database. Each object shown in the schema, such as **STUDENT** or **COURSE**, is known as a **Schema Construct**.

Database Instance (Database State)**Definition**

A **Database Instance**, also called a **Database State** or **Snapshot**, is the actual collection of data stored in the database at a particular point in time. Unlike the schema, the database state changes whenever records are inserted, updated, or deleted.

For example,

Initially,

Student_ID	Name
-------------------	-------------

101	Aisha
-----	-------

After adding another student,

Student_ID	Name
-------------------	-------------

101	Aisha
-----	-------

102	Ahmed
-----	-------

The schema remains unchanged, but the database state has changed.

Characteristics of Database State

- Changes frequently.
- Represents the current contents of the database.
- Updated whenever insert, delete, or update operations are performed.
- Must always satisfy the constraints defined in the schema.

The DBMS is responsible for ensuring that every database state remains **valid**, meaning that it satisfies all structural rules and integrity constraints specified in the schema. The schema descriptions and constraints are stored as **metadata** in the DBMS catalog for reference whenever required.

Schema Evolution

Although a database schema is designed to remain stable, organizational requirements may change over time. When modifications such as adding a new attribute or creating a new table become necessary, the schema itself is updated. This process is known as **Schema Evolution**. Modern DBMSs provide facilities to perform schema evolution while keeping the database operational.

Example:

Adding a new attribute **Date_of_Birth** to the STUDENT table.

Difference Between Schema and Database State

Database Schema	Database State (Instance)
Logical blueprint of the database	Actual data stored in the database
Defined during database design	Changes during database operation
Changes rarely	Changes frequently
Specifies structure	Stores current records
Also called Intension	Also called Extension

A **Data Model** provides the concepts required to represent and manipulate data while achieving data abstraction. Depending on the level of abstraction, data models are classified as conceptual, representational, physical, and self-describing models. The **Database Schema** acts as the logical blueprint of the database, defining its structure and constraints, whereas the **Database State (Instance)** represents the actual data stored at a specific moment in time. Understanding the distinction between schema and instance is essential because the schema remains relatively stable, while the database state changes continuously as users perform insert, update, and delete operations. Together, these concepts form the foundation of database design and management.

2.2 Three-Schema Architecture and Data Independence

Introduction

One of the primary objectives of a Database Management System (DBMS) is to provide **data abstraction**, **program-data independence**, and **multiple user views**. To achieve these objectives, the **Three-Schema Architecture** was proposed. This architecture separates the user's view of the database from the way data is physically stored, making the database easier to design, maintain, and modify. It also provides the foundation for **Data Independence**, allowing changes at one level of the database without affecting other levels.

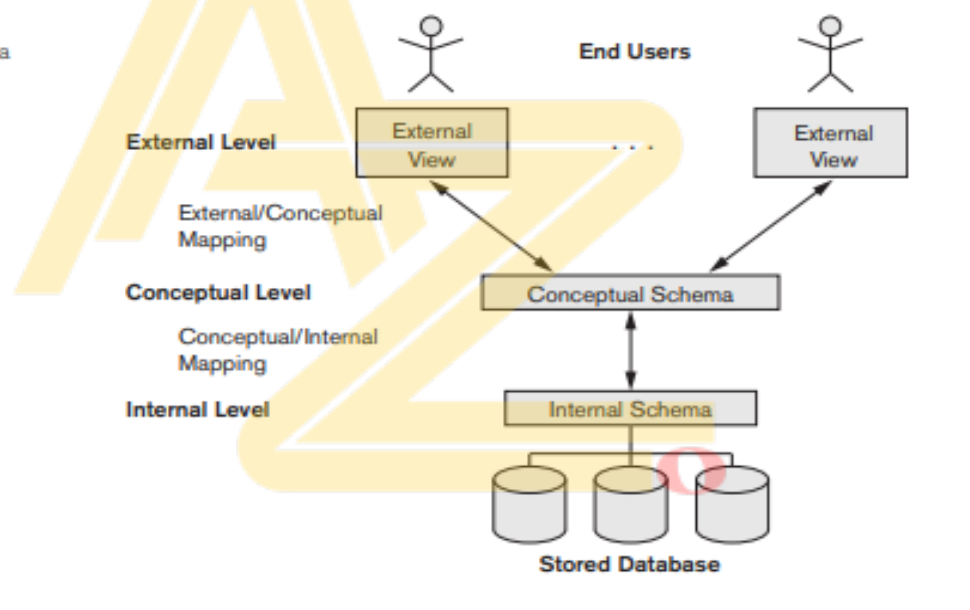
The Three-Schema Architecture was proposed by the **ANSI/SPARC Committee** and is also known as the **ANSI/SPARC Architecture**. Although most modern DBMSs do not implement the architecture exactly as proposed, its concepts are still widely used in database design and form the basis of modern DBMS architecture.

2.2.1 Three-Schema Architecture

Definition

The **Three-Schema Architecture** is a database architecture that separates the database into three different levels of abstraction. Each level provides a different view of the same database and hides unnecessary details from the users at higher levels. The primary goal of this architecture is to separate **user applications from the physical storage of the database**, thereby improving flexibility, security, and data independence.

Figure 2.2
The three-schema architecture.



The architecture consists of the following three levels:

1. Internal Level
2. Conceptual Level
3. External (View) Level

1. Internal Level (Physical Level)

The **Internal Level** is the lowest level of the Three-Schema Architecture. It describes **how the data is actually stored** inside the computer. The internal schema specifies the physical storage structures, file organization, record formats, indexes, access paths, and storage locations of the database. It uses a **Physical Data Model** to describe these storage details.

This level is mainly concerned with performance and storage efficiency rather than user interaction. Since it contains implementation details, it is primarily used by database administrators and system developers rather than end users.

Information Stored at Internal Level

- Physical file organization
- Record formats
- Index structures
- Hashing techniques
- Access paths
- Storage allocation

Purpose

- Efficient storage of data.
- Fast retrieval of records.
- Better utilization of storage devices.

2. Conceptual Level (Logical Level)

The **Conceptual Level** provides a complete logical view of the entire database. It describes **what data is stored** in the database without specifying **how** the data is physically stored. The conceptual schema hides all physical implementation details and focuses only on the logical structure of the database.

The conceptual schema defines:

- Entities
- Attributes
- Relationships

- Data types
- Constraints
- User operations

This level represents the database as a whole and serves as an interface between the internal level and the external level.

Purpose

- Provides the overall logical structure of the database.
- Hides physical storage details.
- Maintains consistency among different user views.

3. External Level (View Level)

The **External Level** is the highest level of the Three-Schema Architecture. It consists of multiple **External Schemas**, also known as **User Views**. Each external schema represents only the portion of the database that is required by a particular user or user group. Information that is irrelevant or confidential is hidden from that user.

Since different users have different requirements, the same database may have several external views.

Examples

- Students view only personal details and examination results.
- Faculty members access student marks and attendance.
- Accounts department views only fee-related information.
- Library staff access book issue and return records.

Thus, every user interacts only with the information relevant to their work.

Advantages

- Improves security.
- Simplifies database usage.
- Supports multiple user requirements.
- Hides unnecessary information.

Mappings in Three-Schema Architecture

The three levels are connected through **Mappings**. A mapping is the process of translating requests and data from one schema level to another. Since users work at the external level while data is stored at the internal level, the DBMS must transform requests through these mappings.

There are two types of mappings:

1. External–Conceptual Mapping

This mapping converts requests from the **External Schema** into the **Conceptual Schema**. It ensures that each user view correctly corresponds to the logical structure of the database.

2. Conceptual–Internal Mapping

This mapping converts requests from the **Conceptual Schema** into the **Internal Schema**, enabling the DBMS to access the actual data stored on the disk.

Working of the Three-Schema Architecture

When a user submits a query, the following sequence of operations takes place:

1. The user issues a query through the **External Schema**.
2. The DBMS converts it into the **Conceptual Schema**.
3. The request is further translated into the **Internal Schema**.
4. The DBMS retrieves the required data from the stored database.
5. The retrieved data is converted back through the mappings.
6. The user receives the information in the required external view.

Thus, the user never interacts directly with the physical storage of the database.

Advantages of Three-Schema Architecture

- Provides data abstraction.
- Supports multiple user views.
- Separates logical and physical storage.

- Simplifies database maintenance.
- Improves security.
- Enables data independence.

2.2.2 Data Independence

Definition

Data Independence is the ability to modify the schema at one level of the database without affecting the schema at the next higher level or the application programs. In other words, changes made to the database structure should not require modifications to existing user programs. This is one of the most important objectives of the Three-Schema Architecture.

Data Independence allows the database to evolve over time while minimizing the impact on applications.

There are two types of Data Independence:

1. Logical Data Independence
2. Physical Data Independence

1. Logical Data Independence

Definition

Logical Data Independence is the ability to modify the **Conceptual Schema** without affecting the **External Schemas** or the application programs that use them.

Changes at the conceptual level may include:

- Adding new tables.
- Adding new attributes.
- Removing unnecessary attributes.
- Modifying constraints.
- Changing relationships.

These modifications should not affect users whose external views remain unchanged.

Example

Suppose the **STUDENT** table initially contains:

- Student_ID
- Name
- Class

Later, a new attribute **Email_ID** is added.

Only the conceptual schema changes. Existing application programs that do not use the Email_ID attribute continue to function without modification.

Advantages

- Existing applications continue to work.
- Simplifies database expansion.
- Reduces software maintenance.
- Supports evolving business requirements.

2. Physical Data Independence

Definition

Physical Data Independence is the ability to modify the **Internal Schema** without affecting the **Conceptual Schema** or application programs.

Changes at the internal level may include:

- Creating new indexes.
- Changing file organization.
- Modifying access paths.
- Changing storage devices.
- Reorganizing physical files.

Since these changes occur only in the physical storage structure, users and application programs remain completely unaffected.

Example

Suppose an index is created on the **SECTION** table to improve search speed.

The logical structure of the database remains unchanged.

Users continue to execute the same SQL queries, but retrieval becomes much faster.

Advantages

- Improves system performance.
- Allows storage optimization.
- No changes to application programs.
- Better utilization of hardware resources.

Difference Between Logical and Physical Data Independence

Logical Data Independence

Changes Conceptual Schema

External Schemas remain unchanged

Difficult to achieve

Involves logical structure

Example: Add a new attribute

Physical Data Independence

Changes Internal Schema

Conceptual Schema remains unchanged

Easier to achieve

Involves physical storage

Example: Create an index

Importance of Data Independence

Data Independence provides several important benefits:

- Reduces application maintenance.
- Allows database modifications without affecting users.
- Improves system flexibility.
- Supports database evolution.
- Simplifies future expansion.
- Enhances database reliability.

The **Three-Schema Architecture** is one of the fundamental concepts in DBMS that separates the database into **External**, **Conceptual**, and **Internal** levels. This separation provides **data abstraction**, supports **multiple user views**, and isolates user applications from physical storage details. The architecture also enables **Data Independence**, allowing changes at the conceptual or internal level without affecting higher levels or application programs. Through **Logical Data Independence**, the conceptual structure of the database can evolve without changing user views, while **Physical Data Independence** allows storage optimization without affecting the logical design. Together, these concepts make database systems flexible, maintainable, and scalable, which is why they remain a cornerstone of modern DBMS design.

2.3 Database Languages and Interfaces

Introduction

A Database Management System (DBMS) is used by different categories of users, such as Database Administrators (DBAs), database designers, application programmers, and end users. Since each user performs different tasks, a DBMS provides different types of **database languages** and **user interfaces** to enable efficient interaction with the database. Database languages are used to define, manipulate, and manage the database, whereas interfaces provide user-friendly methods for accessing database services without requiring detailed knowledge of database programming. Together, these languages and interfaces make a DBMS flexible, efficient, and easy to use for both technical and non-technical users.

The two major components discussed in this section are:

1. **DBMS Languages**
2. **DBMS Interfaces**

2.3.1 DBMS Languages

Introduction

After designing a database and selecting an appropriate DBMS, the next step is to define the database structure and populate it with data. To accomplish these tasks, the DBMS provides several specialized languages. These languages allow users to define database schemas, manipulate stored data, specify user views, and control physical

storage structures. In modern DBMSs, these functions are often integrated into a single comprehensive language such as **SQL (Structured Query Language)**.

The major database languages are:

- Data Definition Language (DDL)
- Storage Definition Language (SDL)
- View Definition Language (VDL)
- Data Manipulation Language (DML)

1. Data Definition Language (DDL)

Definition

A **Data Definition Language (DDL)** is a language used to define the structure of a database. It enables the Database Administrator (DBA) and database designers to create database schemas, specify tables, define attributes, establish relationships, and enforce integrity constraints. When DDL statements are executed, the **DDL Compiler** processes these statements and stores the schema descriptions in the **DBMS Catalog (Metadata Repository)**.

In most modern relational database systems, DDL is also used to define user views and conceptual schemas.

Functions of DDL

- Creates database schemas.
- Defines tables and attributes.
- Specifies relationships.
- Defines integrity constraints.
- Stores metadata in the DBMS catalog.

Examples of DDL Commands

- CREATE
- ALTER
- DROP

- TRUNCATE

2. Storage Definition Language (SDL)

Definition

The **Storage Definition Language (SDL)** is used to specify the **internal schema** of the database. It defines how data is physically stored on secondary storage devices. SDL specifies storage structures, file organization, indexing methods, and access paths that improve database performance.

In modern relational DBMSs, SDL is not available as a separate language. Instead, physical storage is managed using storage parameters, indexing options, and other administrative commands.

Functions of SDL

- Defines physical storage structures.
- Specifies file organization.
- Controls indexing.
- Optimizes storage performance.

3. View Definition Language (VDL)

Definition

A **View Definition Language (VDL)** is used to define **External Schemas (User Views)**. It specifies the portion of the database that is visible to a particular user or user group. By defining customized views, the DBMS hides unnecessary or confidential information from users.

In relational database systems, SQL performs the role of VDL through the **CREATE VIEW** statement.

Functions of VDL

- Creates user views.
- Provides data abstraction.
- Improves security.
- Supports multiple user views.

4. Data Manipulation Language (DML)

Definition

A **Data Manipulation Language (DML)** is used to retrieve and manipulate the data stored in the database. After the database has been created, users interact with the stored information using DML statements. These operations include inserting new records, modifying existing records, deleting unwanted records, and retrieving required information.

Functions of DML

- Insert data.
- Update existing records.
- Delete records.
- Retrieve information.

Examples of DML Commands

- SELECT
- INSERT
- UPDATE
- DELETE

Types of DML

Depending on the method of operation, DML is classified into two types.

a) High-Level (Non-Procedural) DML

A **High-Level DML** allows users to specify **what data is required** without describing how the database should retrieve it. Since users simply describe the desired result, these languages are also called **Declarative Languages**. High-level DML can retrieve multiple records using a single statement and is therefore called **Set-Oriented DML** or **Set-at-a-Time DML**. SQL is the most common example of a high-level DML.

Characteristics

- Easy to learn and use.
- Retrieves multiple records at once.
- Focuses on *what* data is required.
- Used by casual users and database administrators.

b) Low-Level (Procedural) DML

A **Low-Level DML** requires users to specify **how** the data should be retrieved. It retrieves one record at a time and therefore requires programming constructs such as loops and conditions. Because it processes individual records, it is known as **Record-at-a-Time DML**. Low-level DML is generally embedded within programming languages.

Characteristics

- Processes one record at a time.
- Requires programming logic.
- Embedded in programming languages.
- Used by application programmers.

Host Language and Data Sublanguage

When DML statements are embedded inside a general-purpose programming language such as **C, C++, or Java**, the programming language is called the **Host Language**, while the embedded DML is known as the **Data Sublanguage**. This approach allows application programs to interact directly with the database while combining programming logic with database operations.

Query Language

When a high-level DML is used independently to retrieve or update data interactively, it is called a **Query Language**. Query languages allow users to execute database operations directly without writing complete application programs. SQL is the most widely used query language in relational database systems.

Comparison of DDL, SDL, VDL and DML

Language	Purpose	Used By
DDL	Defines database structure	DBA, Database Designer
SDL	Defines physical storage	DBA
VDL	Defines user views	DBA
DML	Manipulates stored data	End Users, Programmers

Comparison of High-Level and Low-Level DML

High-Level DML

Non-Procedural

Specifies *what* to retrieve

Retrieves multiple records

Set-Oriented

Example: SQL

Low-Level DML

Procedural

Specifies *how* to retrieve

Retrieves one record at a time

Record-Oriented

Embedded DML

2.3.2 DBMS Interfaces

Introduction

Different categories of users interact with a database in different ways. While programmers may use SQL commands, casual users often prefer graphical interfaces, and naive users rely on predefined forms or menus. Therefore, a DBMS provides several **user-friendly interfaces** that simplify interaction with the database and eliminate the need to remember complex query syntax. These interfaces improve usability, reduce training requirements, and make database systems accessible to users with different technical backgrounds.

1. Menu-Based Interfaces

Menu-based interfaces present users with a series of menus containing predefined options. Users simply select the required option instead of writing SQL commands.

These interfaces are commonly used in websites and database browsing applications because they are simple and intuitive. Pull-down menus are a popular example of menu-based interfaces.

2. Mobile Application Interfaces

Many organizations provide dedicated mobile applications that allow users to access databases using smartphones or tablets. These applications offer predefined options such as account balance enquiry, reservation booking, bill payment, and insurance services through secure login mechanisms.

3. Forms-Based Interfaces

Forms-based interfaces display structured forms through which users can insert, update, or retrieve information. These interfaces are mainly designed for **naïve users** who repeatedly perform predefined transactions. By filling only selected fields, users can retrieve matching database records without writing SQL queries.

4. Graphical User Interfaces (GUI)

A **Graphical User Interface (GUI)** represents the database visually using diagrams, icons, menus, and forms. Users can formulate database queries by interacting with graphical components instead of typing commands. Modern DBMS software extensively uses GUI-based interfaces because they are highly interactive and user-friendly.

5. Natural Language Interfaces

Natural Language Interfaces allow users to submit requests in ordinary languages such as English. The system interprets the request, converts it into an equivalent database query, and retrieves the required information. If the request is ambiguous, the interface interacts with the user for clarification.

6. Keyword-Based Database Search

Keyword-based search interfaces work similarly to Internet search engines. Users enter keywords instead of SQL queries, and the DBMS searches indexed database contents to retrieve the most relevant results. This approach is becoming increasingly common in modern database applications.

7. Speech Input and Output Interfaces

Speech interfaces allow users to interact with databases through voice commands. The spoken request is converted into a database query, and the retrieved result may also be presented as synthesized speech. These interfaces are commonly used in telephone enquiry systems, banking applications, and customer support services.

8. Interfaces for Parametric Users

Parametric users repeatedly perform the same predefined operations. Their interfaces usually contain buttons, function keys, or menu options that execute standard transactions with minimal user input.

Example:

- Bank Teller System
- Railway Reservation Counter
- Airline Reservation System

9. Interfaces for the Database Administrator (DBA)

The DBA requires specialized interfaces containing privileged administrative commands. These interfaces are used to create user accounts, grant permissions, modify database schemas, manage storage structures, monitor performance, and configure system parameters. Since these operations directly affect the database, access is restricted to authorized administrators only.

A DBMS provides a comprehensive set of **database languages** and **user interfaces** to support different categories of users. Languages such as **DDL**, **SDL**, **VDL**, and **DML** enable database definition, storage management, view creation, and data manipulation. Modern DBMSs generally integrate these functions into a single language such as SQL. To simplify interaction with the database, various interfaces—including menu-based, forms-based, graphical, natural language, keyword-based, speech-enabled, mobile, and administrator interfaces—are provided. Together, these languages and interfaces ensure that both technical and non-technical users can efficiently access, manage, and manipulate the database according to their requirements.

2.4 The Database System Environment

Introduction

A **Database Management System (DBMS)** is not a single program but a **complex software system** consisting of several interconnected modules. Each module performs a specific function, such as processing queries, managing storage, maintaining security, handling transactions, and interacting with users. Together, these modules ensure that the database operates efficiently, securely, and reliably.

Apart from the DBMS software itself, the database environment also includes the **Operating System (OS)**, application programs, utility programs, communication software, and various development tools. These components work together to provide an effective environment for storing, retrieving, and managing data.

The Database System Environment can be studied under the following sections:

1. DBMS Component Modules
2. Database System Utilities
3. Tools, Application Environments, and Communication Facilities

2.4.1 DBMS Component Modules

Introduction

A DBMS consists of several software modules, each responsible for a particular activity within the database system. Some modules interact directly with users, while others manage data storage, query execution, transaction processing, and communication with the operating system. These modules work together to process database requests efficiently.

The DBMS architecture can be divided into two major parts:

- **Upper Layer (User Interfaces)**
- **Lower Layer (Internal Processing Modules)**

Architecture of DBMS Component Modules

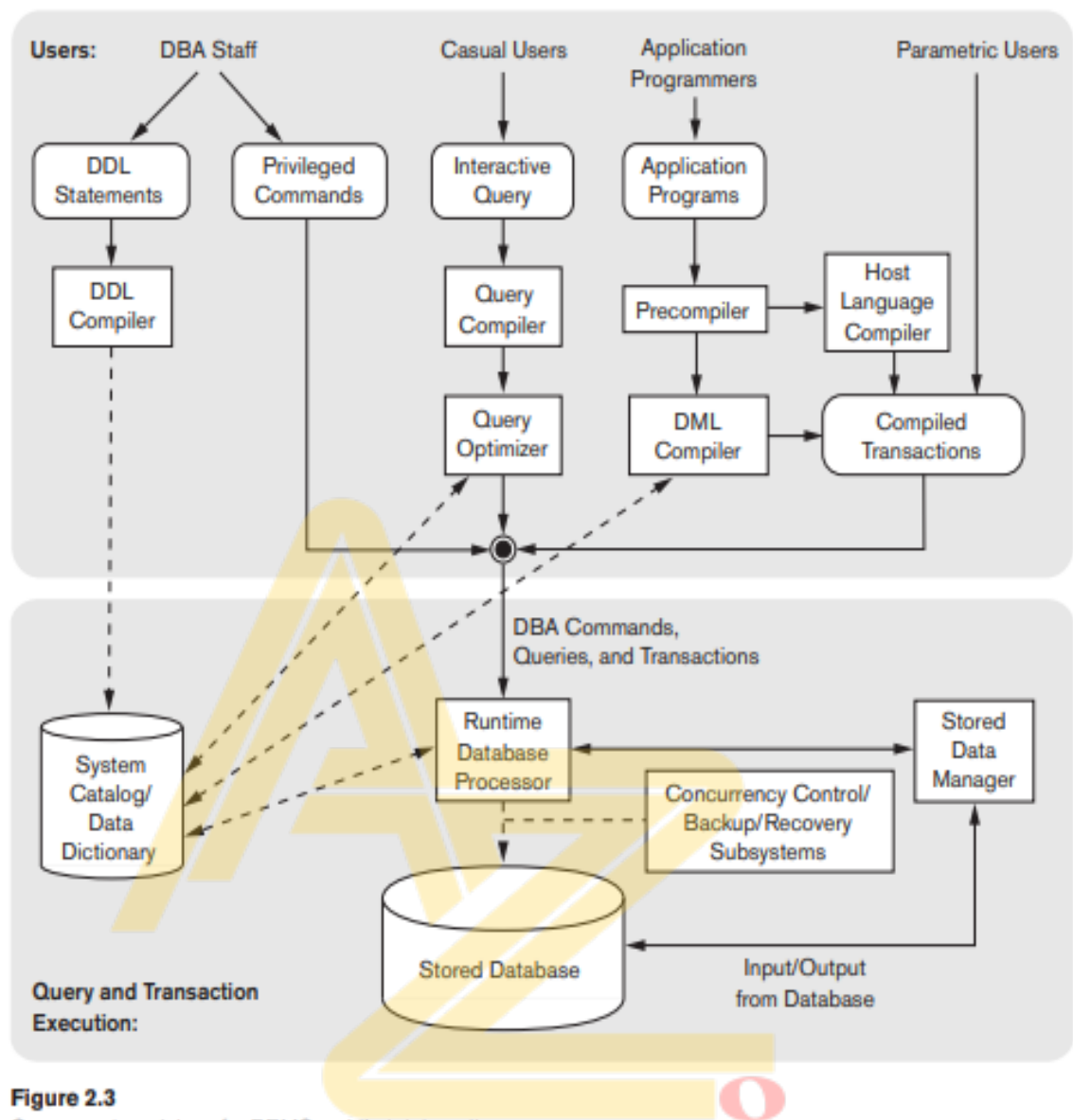


Figure 2.3
Component modules of a DBMS and their interactions.

Users of the Database System

Different categories of users interact with the DBMS using different interfaces.

1. Database Administrator (DBA)

The DBA defines the database schema, manages storage, grants user privileges, and tunes database performance. The DBA primarily uses the **DDL** and privileged administrative commands.

2. Casual Users

Casual users retrieve information occasionally by writing interactive SQL queries or by using graphical interfaces.

Example:

- Managers
- Department Heads
- Business Executives

3. Application Programmers

Application programmers develop database applications using programming languages such as Java, C++, Python, or PHP. These applications contain embedded SQL statements for accessing the database.

4. Parametric Users

Parametric users repeatedly execute predefined transactions by supplying input parameters.

Examples:

- Bank Tellers
- Railway Reservation Clerks
- Cashiers

Major DBMS Modules

1. DDL Compiler

The **Data Definition Language (DDL) Compiler** processes DDL statements issued by the Database Administrator. It converts schema definitions into an internal format and stores the metadata in the **System Catalog (Data Dictionary)**.

Functions

- Processes CREATE, ALTER, and DROP statements.
- Generates metadata.

- Updates the system catalog.

2. System Catalog (Data Dictionary)

The **System Catalog** stores metadata about the database rather than the actual data.

It contains information such as:

- Table names
- Attribute names
- Data types
- Constraints
- File sizes
- Storage details
- Mapping information
- User privileges

The DBMS continuously consults the catalog whenever queries are processed.

3. Query Compiler

The **Query Compiler** processes SQL queries submitted by users. It checks the correctness of the query syntax, verifies table and attribute names, and converts the query into an internal representation suitable for execution.

Functions

- Syntax checking.
- Semantic checking.
- Internal query generation.

4. Query Optimizer

The **Query Optimizer** improves query performance by selecting the most efficient execution plan.

It performs operations such as:

- Reordering operations.

- Eliminating redundant computations.
- Selecting efficient search algorithms.
- Choosing appropriate indexes.

The optimizer consults statistical information stored in the system catalog before generating the final execution plan.

5. Precompiler

When application programs contain embedded SQL statements, the **Precompiler** separates the SQL commands from the host programming language.

The extracted SQL statements are sent to the DML Compiler, while the remaining program is forwarded to the Host Language Compiler.

6. DML Compiler

The **Data Manipulation Language (DML) Compiler** compiles SQL commands such as:

- SELECT
- INSERT
- UPDATE
- DELETE

into executable object code that can access the database efficiently.

7. Host Language Compiler

The **Host Language Compiler** compiles the remaining part of the application program written in programming languages such as:

- Java
- C
- C++
- Python

After compilation, both the application program and compiled SQL statements are linked together to form an executable program.

8. Runtime Database Processor

The **Runtime Database Processor** executes:

- Interactive queries
- Privileged commands
- Predefined transactions

It communicates with the System Catalog and Stored Data Manager while executing database operations. It is responsible for managing runtime execution and data transfer between memory and storage.

9. Stored Data Manager

The **Stored Data Manager** controls access to the stored database and the system catalog.

It interacts with the Operating System to perform:

- Disk read operations.
- Disk write operations.
- Buffer management.
- Storage allocation.

The Stored Data Manager ensures efficient movement of data between secondary storage and main memory.

10. Concurrency Control and Recovery Subsystem

This subsystem ensures that multiple users can access the database simultaneously without causing inconsistencies.

Its major responsibilities include:

- Transaction management.
- Concurrency control.
- Backup.
- Recovery after failures.

These mechanisms ensure that the database remains consistent even during system failures.

Working of DBMS Component Modules

The execution of a database request follows these steps:

1. The user submits a query.
2. The Query Compiler checks the query.
3. The Query Optimizer selects the best execution plan.
4. The Runtime Processor executes the query.
5. The Stored Data Manager retrieves the required data.
6. The Operating System performs disk operations.
7. The result is returned to the user.

For application programs:

1. SQL statements are extracted by the Precompiler.
2. SQL is compiled by the DML Compiler.
3. Program code is compiled by the Host Language Compiler.
4. Both are linked together.
5. The Runtime Processor executes the transaction.

2.4.2 Database System Utilities

Introduction

In addition to its internal processing modules, a DBMS provides several **utility programs** that assist the Database Administrator in maintaining and managing the database efficiently. These utilities automate routine administrative tasks and improve database performance.

1. Loading Utility

The **Loading Utility** transfers existing external data files into the database. It automatically converts data from one format into another and stores it in the required database structure.

It is also useful when transferring data between different DBMS products.

2. Backup Utility

The **Backup Utility** creates duplicate copies of the database.

These backup copies are used to restore the database after:

- Disk failure.
- Hardware failure.
- Software crash.
- Accidental deletion.

Modern DBMSs also support **Incremental Backup**, where only the modified data since the previous backup is stored, thereby reducing storage requirements.

3. Database Storage Reorganization Utility

As databases grow, storage structures may become inefficient.

The **Storage Reorganization Utility** reorganizes database files and creates new access paths to improve overall performance.

4. Performance Monitoring Utility

The **Performance Monitoring Utility** continuously observes database activity and generates performance statistics.

These statistics help the DBA decide whether:

- New indexes should be created.
- Existing indexes should be removed.
- Database files should be reorganized.

This improves query performance and resource utilization.

Other Utilities

Many DBMSs also provide utilities for:

- Data compression
- File sorting

- User access monitoring
- Network management
- Data conversion

2.4.3 Tools, Application Environments, and Communication Facilities

Introduction

Modern DBMSs are supported by several development tools and communication facilities that simplify database design, application development, and remote database access. These tools increase developer productivity and improve database administration.

1. CASE Tools

Computer-Aided Software Engineering (CASE) Tools assist database designers during the database design process.

They help in:

- Database modeling.
- ER diagram creation.
- Schema design.
- Documentation.

CASE tools reduce development time and improve design quality.

2. Data Dictionary (Information Repository)

A **Data Dictionary** is an expanded repository containing metadata and additional information such as:

- Database standards.
- Design decisions.
- User information.
- Application descriptions.
- Usage guidelines.

Unlike the System Catalog, which is mainly used by the DBMS internally, the Data Dictionary is frequently accessed by both users and administrators.

3. Application Development Environments

Application development environments provide integrated tools for developing database applications.

These environments support:

- GUI development.
- Database design.
- Query writing.
- Report generation.
- Application programming.

Examples include:

- PowerBuilder
- JBuilder

4. Communication Facilities

Many users access databases from remote locations through:

- Personal Computers
- Workstations
- Mobile Devices

Communication software enables interaction between:

- Client Computer
- Application Server
- Database Server

When DBMS software is integrated with communication software, the system is known as a **DB/DC (Database/Data Communication) System**. These systems commonly use:

- Local Area Networks (LAN)

- Internet
- Routers
- Satellite Networks

Communication facilities make remote database access possible and support distributed database environments.

The **Database System Environment** consists of a collection of software modules, utility programs, development tools, and communication facilities that work together to manage the database efficiently. Internal modules such as the **DDL Compiler**, **Query Compiler**, **Query Optimizer**, **Runtime Database Processor**, and **Stored Data Manager** handle database processing and storage. Utility programs assist in loading data, performing backups, reorganizing storage, and monitoring performance. Additional tools such as **CASE tools**, **Data Dictionaries**, and **Application Development Environments**, together with communication facilities, provide a complete ecosystem for designing, developing, maintaining, and accessing modern database systems. Collectively, these components ensure that a DBMS delivers secure, reliable, efficient, and scalable database services.

2.5 Centralized and Client/Server Architectures for DBMSs

Introduction

The architecture of a Database Management System (DBMS) has evolved alongside the development of computer technology. Early database systems were designed to operate on **centralized mainframe computers**, where all database processing was performed on a single machine. As hardware became more affordable and powerful, personal computers (PCs), workstations, and mobile devices gained processing capabilities, leading to the development of **Client/Server Architectures**. These architectures distribute database processing between clients and servers, resulting in improved performance, scalability, flexibility, and resource sharing. Today, most modern database applications use either **two-tier**, **three-tier**, or **n-tier client/server architectures** depending on the complexity of the application.

This section discusses:

1. **Centralized DBMS Architecture**
2. **Basic Client/Server Architecture**

3. Two-Tier Client/Server Architecture

4. Three-Tier and n-Tier Architecture

2.5.1 Centralized DBMS Architecture

Introduction

The **Centralized DBMS Architecture** was the earliest architecture used in database systems. In this architecture, **all database processing, application execution, and user interface processing are performed on a single central computer**, usually a mainframe. Users access the database through terminals that have little or no processing capability. These terminals function only as input and output devices, displaying information received from the central computer and sending user requests back to it. Since all computation is performed centrally, this architecture is known as a **Centralized Architecture**.

In earlier systems, computer terminals were connected to the mainframe through communication networks. Whenever a user requested information, the request was processed entirely on the central machine, and only the output was transmitted back to the terminal. As technology advanced, terminals were gradually replaced by PCs, workstations, and mobile devices. Initially, these devices were still used only as display terminals, while the DBMS continued to operate as a centralized system. Later, their processing power was utilized, giving rise to client/server architectures.

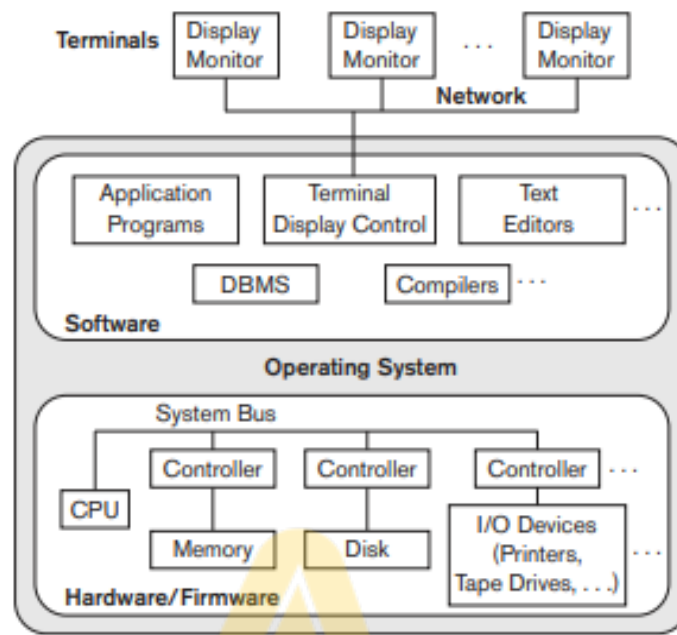


Figure 2.4
A physical centralized architecture.

Components of Centralized Architecture

A centralized database system generally consists of:

- Mainframe or Central Computer
- Database Management System (DBMS)
- Application Programs
- User Interface Programs
- Operating System
- Communication Network
- User Terminals

All these components reside on the same central machine except the display terminals.

Working of Centralized Architecture

The operation of a centralized DBMS follows these steps:

1. The user enters a request through a terminal.
2. The request is transmitted to the central computer.

3. The DBMS processes the request.
4. The required data is retrieved from the database.
5. The processed result is sent back to the user's terminal.

Since all processing occurs at one location, the terminals simply display the output without performing any computation.

Advantages

- Simple system management.
- Centralized control of data.
- Easier security management.
- Easy database maintenance.
- Consistent database environment.

Limitations

- Single point of failure.
- Heavy processing load on the central computer.
- Limited scalability.
- Performance degrades as the number of users increases.
- Users depend entirely on the central system.

These limitations motivated the development of distributed client/server architectures.

2.5.2 Basic Client/Server Architecture

Introduction

As computers became more powerful and affordable, organizations began connecting multiple computers through local area networks (LANs) and wide area networks (WANs). Instead of relying on one central computer, computing tasks were divided between **clients** and **servers**. This approach is known as the **Client/Server Architecture**.

The basic idea behind this architecture is to assign specialized tasks to dedicated servers. Clients request services, while servers process those requests and provide the required resources.

Client

A **Client** is a computer or mobile device that provides the user interface and performs local processing. It requests services such as database access, printing, or file sharing from specialized servers.

Examples include:

- Personal Computers (PCs)
- Workstations
- Laptops
- Smartphones
- Tablets

Server

A **Server** is a computer that provides specialized services to client machines.

Different types of servers include:

- Database Server
- File Server
- Print Server
- Web Server
- Email Server

Each server is responsible for a specific function and can serve multiple clients simultaneously.

Working of Client/Server Architecture

1. The client sends a service request.
2. The request reaches the appropriate server.
3. The server processes the request.

4. The server returns the requested result to the client.

This architecture distributes workload efficiently among different computers.

Advantages

- Better resource sharing.
- Reduced workload on a single computer.
- Improved scalability.
- Better performance.
- Easy maintenance.

2.5.3 Two-Tier Client/Server Architecture for DBMSs

Introduction

The **Two-Tier Architecture** is one of the earliest client/server architectures used in Relational Database Management Systems (RDBMSs). In this architecture, the software components are divided between **two systems**: the **Client** and the **Database Server**. The client handles the user interface and application logic, while the database server manages SQL query processing and transaction execution.

The database server is often called an **SQL Server**, **Query Server**, or **Transaction Server** because it processes SQL queries and transactions submitted by client applications.

Figure 2.5
Logical two-tier
client/server
architecture.

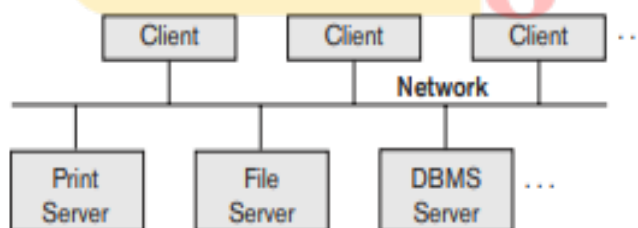
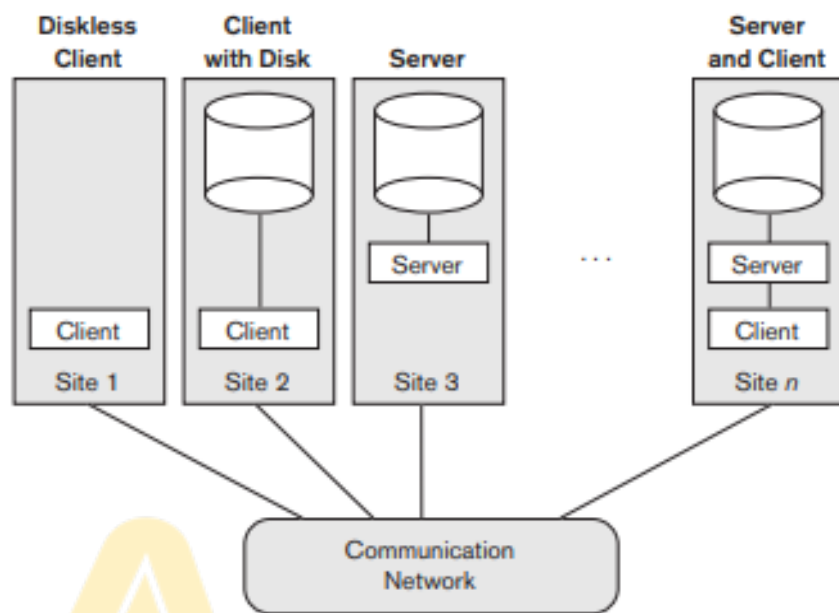


Figure 2.6
Physical two-tier
client/server
architecture.



Working of Two-Tier Architecture

The operation of a two-tier architecture is as follows:

1. The user interacts with the application running on the client.
2. The client establishes a connection with the database server.
3. SQL queries are sent to the server.
4. The server processes the queries.
5. The results are returned to the client.
6. The client formats and displays the output.

Thus, processing responsibilities are shared between the client and the database server.

ODBC (Open Database Connectivity)

To enable communication between client applications and different database systems, Microsoft introduced **Open Database Connectivity (ODBC)**.

ODBC provides a standard **Application Programming Interface (API)** that allows client programs to communicate with different DBMSs using a common interface.

Its advantages include:

- DBMS independence.
- Standard communication.
- Easy database connectivity.
- Supports multiple RDBMS products.

JDBC (Java Database Connectivity)

For Java applications, a similar standard known as **Java Database Connectivity (JDBC)** was introduced.

JDBC enables Java programs to connect to relational databases through a standardized interface, allowing Java applications to interact with different DBMSs without modifying application code.

Advantages

- Simple architecture.
- Easy implementation.
- Good performance for small applications.
- Supports standard SQL communication.
- Compatible with existing systems.

Limitations

- Heavy load on the database server.
- Difficult to support large web applications.
- Limited scalability.
- Business logic remains on the client.

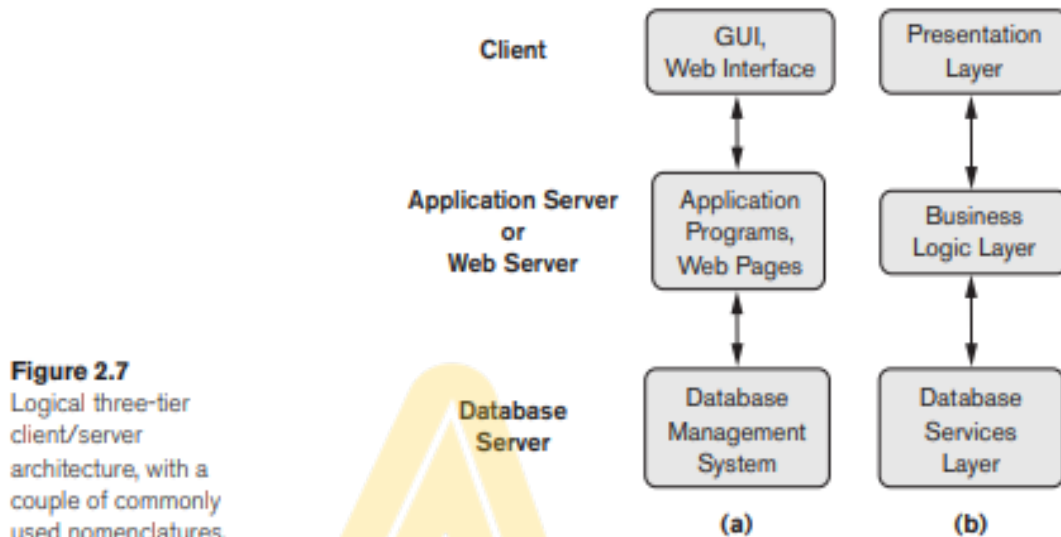
These limitations led to the development of the Three-Tier Architecture.

2.5.4 Three-Tier and n-Tier Architectures for Web Applications

Introduction

Modern web-based applications require better scalability, improved security, and easier maintenance than what the two-tier architecture can provide. To satisfy these requirements, the **Three-Tier Architecture** introduces an additional intermediate layer

called the **Application Server** or **Web Server**. This middle layer separates business logic from both the client and the database server, improving flexibility and performance.



Working of Three-Tier Architecture

The sequence of operations is as follows:

1. The client sends a request.
2. The Application Server receives the request.
3. Business rules are processed.
4. Database queries are sent to the Database Server.
5. The Database Server processes the request.
6. Results are returned to the Application Server.
7. The Application Server formats the output.
8. The final response is sent to the client.

The client never communicates directly with the database server, thereby improving security and reducing database workload.

Three Layers

1. Presentation Layer

This layer provides the user interface.

Functions:

- Displays information.
- Accepts user input.
- Interacts with users.

2. Business Logic Layer

This intermediate layer contains:

- Application programs.
- Business rules.
- Validation procedures.
- Security checks.

It processes client requests before forwarding them to the database server.

3. Database Services Layer

This layer consists of:

- DBMS.
- Database.
- Storage Manager.
- Transaction Processing.

It performs all database operations.

Advantages of Three-Tier Architecture

- Better scalability.
- Improved security.
- Reduced database workload.
- Easier maintenance.
- Better performance.

- Centralized business logic.
- Supports web applications.

n-Tier Architecture

As enterprise applications became increasingly complex, developers further divided the business logic into multiple layers, giving rise to the **n-Tier Architecture**.

In an n-tier architecture, additional layers may include:

- Middleware
- Authentication Layer
- Service Layer
- Business Layer
- Database Layer

This architecture is commonly used in:

- Enterprise Resource Planning (ERP)
- Customer Relationship Management (CRM)
- Large E-commerce Applications

The n-tier architecture provides greater modularity, scalability, maintainability, and flexibility than the three-tier architecture.

Comparison of Architectures

Centralized	Two-Tier	Three-Tier
Single Computer	Client + Database Server	Client + Application Server + Database Server
All processing at one location	Client handles UI	Application server handles business logic
Limited scalability	Moderate scalability	High scalability

Centralized	Two-Tier	Three-Tier
Less secure	Better security	Highest security
Suitable for small systems	Suitable for medium applications	Suitable for web and enterprise applications

The architecture of database systems has evolved from **Centralized DBMSs** to **Client/Server Architectures** to meet the increasing demands of modern applications. The centralized architecture performs all processing on a single machine and is suitable for simple environments. The **Two-Tier Architecture** distributes processing between the client and the database server, making it suitable for small and medium-sized applications. For large-scale web and enterprise applications, the **Three-Tier Architecture** introduces an application server that separates business logic from data management, thereby improving security, scalability, and maintainability. Further enhancements have resulted in **n-Tier Architectures**, which divide application logic into multiple specialized layers to support highly distributed and scalable systems. These architectural advancements have enabled modern DBMSs to efficiently support complex, multi-user, and web-based applications.

2.6 Classification of Database Management Systems (DBMSs)

Introduction

A **Database Management System (DBMS)** can be classified in several ways depending on its characteristics, architecture, and application requirements. Since different organizations have different needs, no single type of DBMS is suitable for every application. Therefore, DBMSs are classified based on criteria such as the **data model**, **number of users**, **distribution of data**, **cost**, **access path**, and **purpose of the system**. Among these, the **data model** is considered the most important criterion because it determines how data is organized, stored, and manipulated within the database. Understanding these classifications helps users select the most appropriate DBMS for a particular application.

The major criteria for classifying DBMSs are:

1. Based on Data Model

2. Based on Number of Users
3. Based on Number of Sites
4. Based on Cost
5. Based on Access Path
6. Based on Purpose

2.6.1 Classification Based on Data Model

Introduction

The **data model** is the primary criterion used for classifying Database Management Systems. A data model defines how data is represented, organized, and manipulated inside a database. Depending on the data model used, different types of DBMSs have evolved to support various application requirements. Modern databases include relational, object-oriented, object-relational, NoSQL, XML, hierarchical, and network database systems.

1. Relational DBMS (RDBMS)

A **Relational Database Management System (RDBMS)** stores data in the form of **tables (relations)** consisting of rows and columns. Each table represents a specific entity, while relationships between tables are established using keys. Relational databases use **SQL (Structured Query Language)** for defining, manipulating, and retrieving data. Because of their simplicity, flexibility, and powerful querying capabilities, relational databases are the most widely used DBMSs in commercial applications.

Characteristics

- Data stored as tables.
- Uses SQL language.
- Supports relationships among tables.
- Provides data independence.
- Supports user views.

Examples

- Oracle

- MySQL
- PostgreSQL
- Microsoft SQL Server

2. Object-Oriented DBMS (OODBMS)

An **Object-Oriented Database Management System (OODBMS)** stores data in the form of **objects** rather than tables. Each object contains both data and the operations (methods) that can be performed on that data. Objects having similar properties and behaviors are grouped into **classes**, and classes are organized into hierarchies. OODBMSs support object-oriented concepts such as **encapsulation**, **inheritance**, **object identity**, and **abstract data types**, making them suitable for applications involving engineering, multimedia, and scientific computing.

Characteristics

- Stores objects instead of tables.
- Supports classes and inheritance.
- Methods are associated with objects.
- Suitable for complex data.

3. Object-Relational DBMS (ORDBMS)

An **Object-Relational Database Management System (ORDBMS)** combines the features of relational databases with object-oriented concepts. It extends the relational model by supporting complex data types, user-defined types, and object-oriented features while retaining SQL compatibility. This approach provides the advantages of both relational and object-oriented databases.

Characteristics

- Combines RDBMS and OODBMS features.
- Supports SQL.
- Supports complex objects.
- Allows user-defined data types.

4. NoSQL (Big Data) DBMS

With the rapid growth of social media, cloud computing, and web applications, traditional relational databases became insufficient for managing extremely large and unstructured datasets. This led to the development of **NoSQL (Not Only SQL)** database systems. NoSQL databases are designed to handle massive volumes of structured, semi-structured, and unstructured data with high scalability and performance.

The four major NoSQL data models are:

a) Key-Value Model

Stores each data item as a **key-value pair**.

- Very fast retrieval.
- Suitable for caching and session management.

b) Document Model

Stores data as **JSON-based documents**.

- Flexible structure.
- Suitable for web applications.

c) Graph Model

Stores data using **nodes** and **edges**.

- Efficiently represents relationships.
- Used in social networks and recommendation systems.

d) Column-Based Model

Stores data column-wise rather than row-wise.

- Optimized for analytical processing.
- Supports multiple versions of data.

5. XML DBMS

An **XML Database Management System** stores data in the **XML (eXtensible Markup Language)** format. XML represents information as hierarchical tree structures using nested tags. XML combines concepts from document management systems with

database concepts and is widely used for exchanging information over the Internet. Many commercial DBMSs provide XML support in addition to relational storage.

Characteristics

- Tree-structured model.
- Uses XML tags.
- Suitable for web data exchange.

6. Hierarchical DBMS

A **Hierarchical Database Management System** organizes data in the form of a **tree structure**, where each parent record can have multiple child records. This model represents one-to-many (1:N) relationships and was widely used between **1965 and 1985**. One of the most popular hierarchical database systems was **IMS (Information Management System)**, which used the **DL/1** language.

Characteristics

- Tree structure.
- Parent-child relationship.
- Supports one-to-many relationships.

7. Network DBMS

A **Network Database Management System** represents data using **record types** connected through pointer-based links. Unlike the hierarchical model, a network model allows one record to be associated with multiple parent records, thereby supporting more complex relationships. The network model was standardized by the **CODASYL Database Task Group (DBTG)** and used a record-at-a-time Data Manipulation Language (DML).

Characteristics

- Pointer-based structure.
- Supports one-to-many relationships.
- More flexible than hierarchical model.

2.6.2 Classification Based on Number of Users

A DBMS can also be classified according to the number of users it supports simultaneously.

1. Single-User DBMS

A **Single-User DBMS** allows only **one user** to access the database at a time. These systems are commonly used on personal computers for small-scale applications.

Example

- Microsoft Access

2. Multi-User DBMS

A **Multi-User DBMS** allows **multiple** users to access the database simultaneously. It uses concurrency control techniques to ensure that simultaneous transactions do not cause inconsistencies. Most **commercial** database systems belong to this category.

2.6.3 Classification Based on Number of Sites

Another important criterion is the **physical distribution** of the database.

1. Centralized DBMS

A **Centralized DBMS** stores the complete database at a **single computer site**. Both the DBMS software and the database reside at **one** location, although multiple users may access it remotely.

2. Distributed DBMS (DDBMS)

A **Distributed Database Management System (DDBMS)** distributes both the database and the DBMS software across multiple interconnected sites. Data may be replicated at several locations to improve availability and fault tolerance. Large-scale systems and Big Data environments commonly use distributed architectures.

Types of Distributed DBMS

a) Homogeneous DDBMS

Uses the **same DBMS software** at every site.

b) Heterogeneous DDBMS

Different sites use **different DBMS software**.

c) Federated (Multidatabase) System

A **Federated DBMS** integrates multiple autonomous databases managed by different DBMSs through middleware while allowing each database to maintain local control.

2.6.4 Classification Based on Cost

The cost of a DBMS varies depending on licensing, features, and support services.

1. Open Source DBMS

Open-source DBMSs are available free of cost, although commercial support may be provided by third-party vendors.

Examples

- MySQL
- PostgreSQL

2. Commercial DBMS

Commercial database systems require software licenses and may include additional costs for advanced features such as replication, distributed processing, mobile support, and data warehousing. Large enterprise installations may require significant annual investment for licensing and maintenance.

2.6.5 Classification Based on Access Path

Some DBMSs are classified according to the storage structures and access methods used to retrieve data.

One important example is the **Inverted File DBMS**, which organizes data using inverted file structures to improve retrieval performance for certain types of applications.

2.6.6 Classification Based on Purpose

Depending on the intended application, DBMSs may be classified as **General-Purpose** or **Special-Purpose** systems.

1. General-Purpose DBMS

A **General-Purpose DBMS** is designed to support a wide variety of applications. It provides flexibility, scalability, and support for multiple types of users and applications.

2. Special-Purpose DBMS

A **Special-Purpose DBMS** is designed specifically for a particular application where performance is the primary objective. Such systems are highly optimized but cannot easily be adapted to other applications.

Examples include:

- Airline Reservation Systems
- Telephone Directory Systems

These systems are commonly classified as **Online Transaction Processing (OLTP)** systems because they process a large number of concurrent transactions with minimal delay.

Comparison of Major DBMS Types

Type	Data Representation	Main Feature
Relational	Tables	SQL support
Object-Oriented	Objects	Supports inheritance and methods
Object-Relational	Tables + Objects	Combines relational and object concepts
NoSQL	Key-Value, Document, Graph, Column	Handles Big Data
XML	Tree Structure	Web data exchange
Hierarchical	Tree	Parent-child relationship
Network	Records with pointers	Flexible relationships

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and
student-focused resources for every semester.

Connect With Us:

 www.azdocuments.in [[Click Here](#)]

 [@azdocuments](#) [[Click Here](#)]

 Join Our Student Community: [[Click Here](#)]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.